

第2章 PCI总线的桥与配置

在 PCI 体系结构中，含有两类桥，一类是 HOST 主桥，另一类是 PCI 桥。在每一个 PCI 设备中（包括 PCI 桥）都含有一个配置空间。这个配置空间由 HOST 主桥管理，而 PCI 桥可以转发来自 HOST 主桥的配置访问。在 PCI 总线中，PCI Agent 设备使用的配置空间与 PCI 桥使用的配置空间有些差别，但这些配置空间都是由处理器通过 HOST 主桥管理的。

2.1 存储器域与 PCI 总线域

HOST 主桥的实现因处理器系统而异。PowerPC 处理器和 x86 处理器的 HOST 主桥除了集成方式不同之外，其实现机制也有较大差异。但是这些 HOST 主桥所完成的最基本功能依然是分离存储器域与 PCI 总线域，完成 PCI 总线域到存储器域，存储器域到 PCI 总线域之间的数据传递，并管理 PCI 设备的配置空间。

之前曾经多次提到，在一个处理器系统中，存在 PCI 总线域与存储器域，深入理解这两个域的区别是理解 HOST 主桥的关键所在。在一个处理器系统中，存储器域、PCI 总线域与 HOST 主桥的关系如图 2-1 所示。

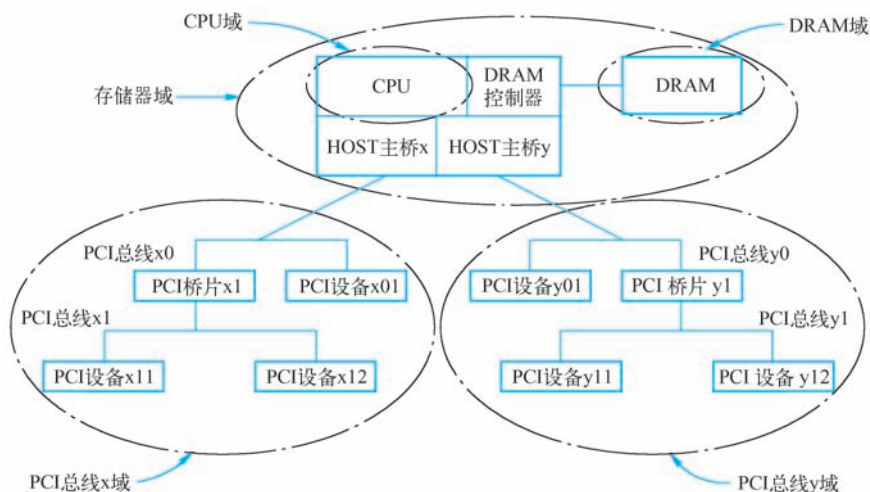


图 2-1 存储器域、PCI 总线域与 HOST 主桥的关系

图中的处理器系统由一个 CPU、一个 DRAM 控制器和两个 HOST 主桥组成。在这个处理器系统中，包含 CPU 域、DRAM 域、存储器域和 PCI 总线域地址空间。其中 HOST 主桥 x 和 HOST 主桥 y 分别管理 PCI 总线 x 域与 PCI 总线 y 域。PCI 设备访问存储器域时，也需要通过 HOST 主桥，并由 HOST 主桥进行 PCI 总线域到存储器域的地址转换；CPU 访问 PCI 设备时，同样需要通过 HOST 主桥进行存储器域到 PCI 总线域的地址转换。

如果 HOST 主桥支持 Peer-to-Peer 传送机制, PCI 总线 x 域上的设备可以与 PCI 总线 y 域上的设备直接通信, 如 PCI 设备 x11 可以直接与 PCI 设备 y11 通信。为简化模型, 在本书中, PCI 总线仅使用 32 位地址空间。

2.1.1 CPU 域、DRAM 域与存储器域

CPU 域地址空间指 CPU 所能直接访问的地址空间集合。在本书中, CPU、处理器与处理器系统的概念不同。如 MPC8548 处理器的内核是 E500 V2^①, 本书将这个处理器内核称为 CPU; 处理器由一个或者多个 CPU、外部 Cache、中断控制器和 DRAM 控制器组成; 而处理器系统由一个或者多个处理器和外部设备组成。

在 CPU 域中有一个重要概念, 即 CPU 域边界。所谓 CPU 域边界, 即 CPU 所能控制的数据完整性边界。CPU 域的边界由 Memory Fence 指令^②的作用范围确定, CPU 域边界的划分对数据完整性 (Data Consistency) 非常重要。与 CPU 域相关的数据完整性知识较为复杂, 可以独立成书, 因此本书对数据完整性不做进一步介绍。

严格地讲, CPU 域仅在 CPU 内核中有效。CPU 访问主存储器时, 首先将读写命令放入读写指令缓冲中, 然后将这个命令发送到 DRAM 控制器或者 HOST 主桥。DRAM 控制器或者 HOST 主桥将 CPU 地址转换为 DRAM 或者 PCI 总线地址, 分别进入 DRAM 域或者 PCI 总线域后, 再访问相应的地址空间。

DRAM 域地址空间指 DRAM 控制器所能访问的地址空间集合。目前处理器系统的 DRAM 一般由 DDR-SDRAM 组成, 有的书籍也将这部分内存称为主存储器。在有些处理器系统中, DRAM 控制器能够访问的地址空间, 并不能被处理器访问, 因此在这类处理器系统中, CPU 域与 DRAM 域地址空间并不等同。

比如有些 CPU 可以支持 36 位的物理地址, 而有些 DRAM 控制器仅支持 32 位的物理地址, 此时 CPU 域包含的地址空间大于 DRAM 域地址空间。但是这并不意味着 DRAM 域一定包含在 CPU 域中, 在某些处理器系统中, CPU 并不能访问在 DRAM 域中的某些数据区域。而 CPU 域中除了包含 DRAM 域外, 还包含外部设备空间。

在多数处理器系统中, DRAM 域空间是 CPU 域空间的一部分, 但是也有例外。比如显卡控制器可能会借用一部分主存储器空间, 这些被借用的空间不能被 CPU 访问, 而只能被 DRAM 控制器, 更为准确地说是显卡通过 DRAM 控制器访问, 因此这段空间不属于 CPU 域, 严格地讲, 这段空间属于外部设备域。

本书使用存储器域统称 CPU 域与 DRAM 域。存储器域包括 CPU 内部的通用寄存器、存储器映像寻址的寄存器、主存储器空间和外部设备空间。在 Intel 的 x86 处理器系统中, 外部设备空间与 PCI 总线域地址空间等效, 因为在 x86 处理器系统中, 使用 PCI 总线统一管理全部外部设备。为简化起见, 本书使用 PCI 总线域替代外部设备域。

值得注意的是, 存储器域的外部设备空间, 在 PCI 总线域中还有一个地址映射。当处理器访问 PCI 设备时, 首先访问的是这个设备在存储器域上的 PCI 设备空间, 之后 HOST 主桥

① MPC8548 处理器基于 E500 V2 内核。目前 E500 内核包括 V1、V2 和 mc (MultiCore) 三个版本。

② x86 处理器的 Memory Fence 指令为 MFENCE、LFENCE 和 SFENCE, 而 PowerPC 处理器的 Memory Fence 指令为 msync 和 mbar。

将这个存储器域的 PCI 总线地址转换为 PCI 总线域的物理地址[⊖]，然后通过 PCI 总线事务访问 PCI 总线域的地址空间。

2.1.2 PCI 总线域

在 x86 处理器系统中，PCI 总线域是外部设备域的重要组成部分。实际上在 Intel 的 x86 处理器系统中，所有的外部设备都使用 PCI 总线管理。而 AMD 的 x86 处理器系统中还存在一条 HT (Hyper Transport) 总线，HT 总线的主要目的是替代 FSB 总线，但是也可以作为局部总线连接一些高速设备即 HT 设备，因此在 AMD 的 x86 处理器系统中还存在 HT 总线域。本书对 HT 总线不做进一步介绍。

PCI 总线域 (PCI Segment) 由 PCI 设备所能直接访问的地址空间组成。在一个处理器系统中，可能存在多个 HOST 主桥，因此也存在多个 PCI 总线域。如在图 2-1 所示的处理器系统中，具有两个 HOST 主桥，因而在这个处理器系统中存在 PCI 总线 x 和 y 域。

在多数处理器系统中，分属于两个 PCI 总线域的 PCI 设备并不能直接进行数据交换，而需要通过 FSB 进行数据交换。值得注意的是，如果某些处理器的 HOST 主桥支持 Peer-to-Peer 数据传送，那么这个 HOST 主桥可以支持不同 PCI 总线域间的数据传送。

PowerPC 处理器使用了 OCeaN 技术连接两个 HOST 主桥，OCeaN 可以将属于 x 域的 PCI 数据请求转发到 y 域，OCeaN 支持 PCI 总线的 Peer-to-Peer 数据传送。有关 OCeaN 技术的详细说明见第 2.2 节。

2.1.3 处理器域

处理器域是指一个处理器系统能够访问的地址空间集合。处理器系统能够访问的地址空间由存储器域和外部设备域组成。其中存储器域地址空间较为简单，而在不同的处理器系统中，外部设备域的组成结构并不相同。如在 x86 处理器系统中，外部设备域主要由 PCI 总线域组成，因为大多数外部设备都是挂接在 PCI 总线[⊖]上的，而在 PowerPC 处理器和其他处理器系统中，有相当多的设备与 FSB 直接相连，而不与 PCI 总线相连。

本书仅介绍 PCI 总线域而不对其他外部设备域进行说明。其中存储器域与 PCI 总线域之间由 HOST 主桥联系在一起。深入理解这些域的关系是深入理解 PCI 体系结构的关键所在，实际上这也是理解处理器体系结构的基础。

通过 HOST 主桥，处理器系统可以将处理器域划分为存储器域与 PCI 总线域。其中存储器域与 PCI 总线域彼此独立，并通过 HOST 主桥进行数据交换。HOST 主桥是联系存储器域与 PCI 总线域的桥梁，是 PCI 总线域实际的管理者。

有些书籍认为 HOST 处理器是 PCI 总线域的管理者，这种说法并不准确。假设在一个 SMP 处理器系统中，存在 4 个 CPU 而只有一个 HOST 主桥，这 4 个 CPU 将无法判断究竟谁是 HOST 处理器。不过究竟是哪个处理器作为 HOST 处理器并不重要，因为在一个处理器系统中，是 HOST 主桥管理 PCI 总线域，而不是 HOST 处理器。当一个处理器系统中含有多个 CPU 时，如果每个 CPU 都可以访问 HOST 主桥，那么这些 CPU 都可以作为这个 HOST 主桥

⊖ PCI 总线域只含有物理地址，因此下文将直接使用 PCI 总线地址，而不使用 PCI 总线物理地址。

⊖ AMD 的 x86 处理器中的某些外部设备，可能是基于 HT 总线，而不使用 PCI 总线。

所管理 PCI 总线树的 HOST 处理器。

在一个处理器系统中, CPU 所能访问的 PCI 总线地址一定在存储器域中具有地址映射; 而 PCI 设备能访问的存储器域的地址也一定在 PCI 总线域中具有地址映射。当 CPU 访问 PCI 域地址空间时, 首先访问存储器域的地址空间, 然后经过 HOST 主桥转换为 PCI 总线域的地址, 再通过 PCI 总线事务进行数据访问。而当 PCI 设备访问主存储器时, 首先通过 PCI 总线事务访问 PCI 总线域的地址空间, 然后经过 HOST 主桥转换为存储器域的地址后, 再对这些空间进行数据访问。

由此可见, 存储器域与 PCI 总线域的转换关系由 HOST 主桥统一进行管理。有些处理器提供了一些寄存器进行这种地址映射, 如 PowerPC 处理器使用 Inbound 和 Outbound 寄存器组保存存储器域与 PCI 总线域的地址映射关系; 而有些处理器并没有提供这些寄存器, 但是存储器域到 PCI 总线域的转换关系依然存在。

HOST 主桥进行不同地址域间的数据交换时, 需要遵循以下规则。为区别存储器域到 PCI 总线域的地址映射, 下面将 PCI 总线域到存储器域的地址映射称为反向映射。

1) 处理器访问 PCI 总线域地址空间时, 首先需要访问存储器域的地址空间, 再通过 HOST 主桥将存储器地址转换为 PCI 总线地址, 之后才能进入 PCI 总线域进行数据交换。PCI 设备使用的地址空间保存在各自的 PCI 配置寄存器中, 即 BAR 寄存器中。这些 PCI 总线地址空间需要在初始化时映射成为存储器域的存储器地址空间, 之后处理器才能访问这些地址空间。在有些处理器的 HOST 主桥中, 具有独立的寄存器保存这个地址映射规则, 如 PowerPC 处理器的 Outbound 寄存器组; 而有些处理器, 如在 x86 处理器中, 虽然没有这样的寄存器组, 但是在 HOST 主桥的硬件逻辑中仍然存在这个地址转换的概念。

2) PCI 设备访问存储器域时, 首先需要访问 PCI 总线域的地址空间, 再通过 HOST 主桥将 PCI 总线地址转换为存储器地址, 才能穿越 HOST 主桥进行数据交换。为此处理器需要通过 HOST 主桥将这个 PCI 总线地址反向映射为存储器地址。PCI 设备不能访问在 PCI 总线域中没有进行这种反向映射的存储器域地址空间。PowerPC 处理器使用 Inbound 寄存器组存放 PCI 设备所能访问的存储器空间, 而在 x86 处理器中并没有这样的寄存器组, 但是依然存在这个地址转换的概念。

3) 如果 HOST 主桥不支持 Peer-to-Peer 传送方式, 那么分属不同 PCI 总线域的 PCI 设备间不能直接进行数据交换。在 32 位的 PCI 总线中, 每一个 PCI 总线域的地址范围都是 0x0000-0000~0xFFFF-FFFF, 但是这些地址没有直接联系。PCI 总线 x 域上的 PCI 总线地址 0x0000-0000 与 PCI 总线 y 域上的 PCI 总线地址 0x0000-0000 并不相同, 而且这两个 PCI 总线地址经过 HOST 主桥反向映射后, 得到的存储器地址也不相同。

在第 2.2 节中, 将主要以 PowerPC 处理器为例说明 HOST 主桥的实现机制, 并在第 2.2.4 节简要说明 x86 处理器中的南北桥构架。尽管部分读者对 PowerPC 处理器并不感兴趣, 笔者仍然强烈建议读者仔细阅读第 2.2 节的全部内容。

在 PowerPC 处理器中, HOST 主桥的实现比较完整, 尤其是 PCI 总线域与存储器域的映射关系比较明晰, 便于读者准确掌握这个重要的概念。而 x86 处理器由于考虑向前兼容 (forward compatibility), 设计中包含了太多的不得已。x86 处理器有时不得不保留原设计中的不完美, 向前兼容是 Intel 的重要成就, 也是一个沉重的十字架。

2.2 HOST 主桥

本节以 MPC8548 处理器为例说明 HOST 主桥在 PowerPC 处理器中的实现机制，并简要介绍 x86 处理器系统使用的 HOST 主桥。

MPC8548 处理器是 Freescale 基于 E500 V2 内核的一个 PowerPC 处理器，该处理器中集成了 DDR 控制器、多个 eTSEC (Enhanced Three-Speed Ethernet Controller)、PCI/PCI-X 和 PCIe 总线控制器等一系列接口。MPC8548 处理器的拓扑结构如图 2-2 所示。

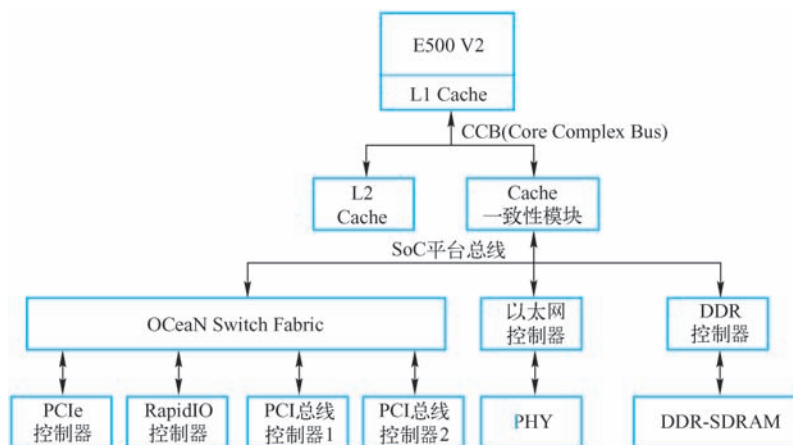


图 2-2 MPC8548 处理器的拓扑结构

图中，MPC8548 处理器的 L1 Cache 在 E500 V2 内核中，而 L2 Cache 与 FSB^①直接相连，不属于 E500 内核。值得注意的是有些高端 PowerPC 处理器的 L2 Cache 也在 CPU 中，而 L3 Cache 与 CCB 总线直接相连。

在 MPC8548 处理器中，所有外部设备，如以太网控制器、DDR 控制器和 OCeaN 连接的总线控制器都与 SoC 平台总线^②直接连接。而 SoC 平台总线通过 Cache 共享一致性模块与 FSB 连接。

在 MPC8548 处理器中，具有一个 32 位的 PCI 总线控制器、一个 64 位的 PCI/PCI-X 总线控制器，还有多个 PCIe 总线控制器。MPC8548 处理器使用 OCeaN 连接这些 PCI、PCI-X 和 PCIe 总线控制器。在 MPC8548 处理器系统中，PCI 设备进行 DMA 操作时，首先通过 OCeaN，之后经过 SoC 平台总线到达 DDR 控制器。

OCeaN 是 MPC8548 处理器中连接快速外设使用的交叉互连总线，不仅可以连接 PCI、PCI-X 和 PCIe 总线控制器，而且可以连接 RapidIO^③总线控制器。使用 OCeaN 进行互连的总线控制器可以直接通信，而不需要通过 SoC 平台总线。

① MPC8548 也将 FSB 称为 CCB (Core Complex Bus)。

② PowerPC 处理器并没有公开其 SoC 平台总线的设计规范。ARM 提出的 AMBA 总线是一条典型的 SoC 平台总线。

③ RapidIO 总线是由 Mercury Computer System 和 Motorola Semiconductor (目前的 Freescale) 共同提出，用于解决背板互连的一条外部总线。

如来自 HOST 主桥 1 的数据报文可以通过 OCeaN 直接发向 HOST 主桥 2，而不需要将数据通过 SoC 平台总线，再进行转发，从而减轻了 SoC 平台总线的负担。OCeaN 部件的拓扑结构如图 2-3 所示。

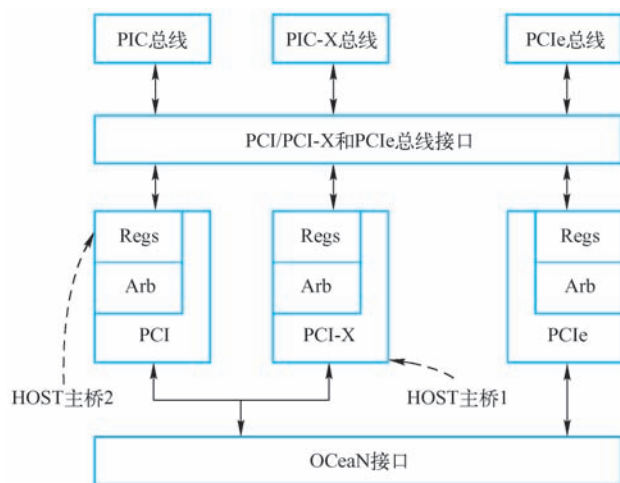


图 2-3 MPC8548 处理器的 HOST 主桥

在 MPC8548 处理器中，有两个 HOST 主桥，分别是 HOST 主桥 1 和 HOST 主桥 2，其中 HOST 主桥 1 可以支持 PCI-X 总线，而 HOST 主桥 2 只能支持 PCI 总线。此外该处理器还含有多个 PCIe 总线控制器。

本节仅介绍 HOST 主桥，即 MPC8548 处理器中的 PCI 总线控制器，而不介绍该处理器的 PCIe 总线控制器。因为从软件层面上看，MPC8548 处理器的 PCIe 总线控制器与 PCI/PCI-X 总线控制器功能类似。

MPC8548 处理器既可以作为 PCI 总线的 HOST 处理器，也可以作为 PCI 总线的从设备，本节仅讲述 MPC8548 处理器如何作为 PCI 总线的 HOST 处理器管理 PCI 总线树，而并不关心 MPC8548 处理器作为从设备的情况。

在 MPC8548 处理器的 HOST 主桥中，定义了一系列与系统软件相关的寄存器。本节将通过介绍这些寄存器，说明这个 HOST 主桥的功能。囿于篇幅，本节仅介绍与 HOST 主桥 1 相关的寄存器，HOST 主桥 2 使用的寄存器与 HOST 主桥 1 使用的寄存器类似。

2.2.1 PCI 设备配置空间的访问机制

PCI 总线规定访问配置空间的总线事务，即配置读写总线事务，使用 ID 号进行寻址。PCI 设备的 ID 号由总线号（Bus Number）、设备号（Device Number）和功能号（Function Number）组成。

其中总线号在 HOST 主桥遍历 PCI 总线树时确定。PCI 总线可以使用 PCI 桥扩展 PCI 总线，并形成一棵 PCI 总线树。在一棵 PCI 总线树上，有几个 PCI 桥（包括 HOST 主桥），就有几条 PCI 总线。在一棵 PCI 总线树中，总线号由系统软件决定，通常与 HOST 主桥直接相连的 PCI 总线编号为 0，系统软件使用 DFS（Depth-First Search）算法扫描 PCI 总线树上的所有 PCI 总线，并依次进行编号。

一条 PCI 总线的设备号由 PCI 设备的 IDSEL 信号与 PCI 总线地址线的连接关系确定，

而功能号与 PCI 设备的具体设计相关。在一个 PCI 设备中最多有 8 个功能设备，而且每一个功能设备都有各自的 PCI 配置空间，而在绝大多数 PCI 设备中只有一个功能设备。HOST 主桥使用寄存器号，访问 PCI 设备配置空间的某个寄存器。

在 MPC8548 处理器的 HOST 主桥中，与 PCI 设备配置空间相关的寄存器由 CFG_ADDR、CFG_DATA 和 INT_ACK 寄存器组成。系统软件使用 CFG_ADDR 和 CFG_DATA 寄存器访问 PCI 设备的配置空间，而使用 INT_ACK 寄存器访问挂接在 PCI 总线上的中断控制器的中断向量，HOST 主桥这 3 个寄存器的地址偏移和属性如表 2-1 所示。

表 2-1 PCI 总线配置寄存器

地 址 偏 移	寄 存 器	属 性	复 位 值
0x0_8000	CFG_ADDR	可读写	0x0000-0000
0x0_8004	CFG_DATA	可读写	0x0000-0000
0x0_8008	INT_ACK	只读	0x0000-0000

在 MPC8548 处理器中，所有内部寄存器都使用存储器映射方式进行寻址，并存放在以 BASE_ADDR[⊖] 变量为起始地址的“1 MB 连续的物理地址空间”中。PowerPC 处理器可以通过 BASE_ADDR+Offset 的方式访问表 2-1 中的寄存器。

MPC8548 处理器使用 CFG_ADDR 寄存器和 CFG_DATA 寄存器访问 PCI 设备的配置空间，其中用 CFG_ADDR 寄存器保存 PCI 设备的 ID 号和寄存器号，该寄存器的各个字段的详细说明如下所示。

- Enable 位。当该位为 1 时，HOST 主桥使能对 PCI 设备配置空间的访问，当 HOST 处理器对 CFG_DATA 寄存器进行访问时，HOST 主桥将对这个寄存器的访问转换为 PCI 配置读写总线事务并发送到 PCI 总线上。
- Bus Number 字段记录 PCI 设备所在的总线号。
- Device Number 字段记录 PCI 设备的设备号。
- Function Number 字段记录 PCI 设备的功能号。
- Register Number 字段记录 PCI 设备的配置寄存器号。

MPC8548 处理器访问 PCI 设备的配置空间时，首先需要在 CFG_ADDR 寄存器中设置这个 PCI 设备对应的总线号、设备号、功能号和寄存器号，然后使能 Enable 位。之后当 MPC8548 处理器对 CFG_DATA 寄存器进行读写访问时，HOST 主桥将这个存储器读写访问转换为 PCI 配置读写请求，并发送到 PCI 总线上。如果 Enable 位没有使能，处理器对 CFG_DATA 的访问不过是一个普通的 I/O 访问，HOST 主桥并不能将其转换为 PCI 配置读写请求。

HOST 主桥根据 CFG_ADDR 寄存器中的 ID 号，生成 PCI 配置读写总线事务，并将这个读写总线事务，通过 ID 译码方式发送到指定的 PCI 设备。PCI 设备将接收来自配置写总线事务的数据，或者为配置读总线事务提供数据。

值得注意的是，在 PowerPC 处理器中，在 CFG_DATA 寄存器中保存的数据采用大端方式进行编址，而 PCI 设备的配置寄存器采用小端编址，因此 HOST 主桥需要进行端模式转换。下面以源代码 2-1 为例说明 PowerPC 处理器如何访问 PCI 配置空间。

⊖ 在 MPC8548 处理器中，BASE_ADDR 存放在 CCSRBAR 寄存器中。

源代码 2-1 PowerPC 处理器访问 PCI 配置空间

```
stw r0,0(r1)
ld r3,0(r2)
```

首先假设寄存器 r1 的初始值为 BASE_ADDR+0x0_8000（即 CFG_ADDR 寄存器的地址），寄存器 r0 的初始值为 0x8000-0008，寄存器 r2 的初始值为 BASE_ADDR+0x0_8004（即 CFG_DATA 寄存器的地址），而指定 PCI 设备（总线号、设备号、功能号都为 0）的配置寄存器的 0x0B~0x08 中的值为 0x9988-7766。

这段源代码的执行步骤如下。

1) 将 r0 寄存器赋值到 r1 寄存器所指向的地址空间中，即初始化 CFG_ADDR 寄存器为 0x8000-0008。

2) 从 r2 寄存器所指向的地址空间中读取数据到 r3 寄存器中，即从 CFG_DATA 寄存器中读取数据到 r3 寄存器。

在 MPC8548 处理器中，源代码 2-1 执行完毕后，寄存器 r3 保存的值为 0x6677-8899，而不是 0x9988-7766。系统程序员在使用这个返回值时，一定要注意大小端模式的转换。值得注意的是，源代码 2-1 可以使用 lwbx 指令进行优化，该指令可以在读取数据的同时，进行大小端模式的转换。

处理器读取 INT_ACK 寄存器时，HOST 主桥将这个读操作转换为 PCI 总线中断响应事务。PCI 总线中断响应事务的作用是通过 PCI 总线读取中断控制器的中断向量号，这样做的前提是中断控制器需要连接在 PCI 总线上。

PowerPC 处理器使用的 MPIC 中断控制器不是挂接在 PCI 总线上，而是挂接在 SoC 平台总线上，因此 PCI 总线提供的中断应答事务在这个处理器系统中并没有太大用途。但是并不排除某些 PowerPC 处理器系统使用了挂接在 PCI 总线上的中断控制器，比如 PCI 南桥芯片，此时 PowerPC 处理器系统需要使用中断应答事务读取 PCI 南桥中的中断控制器，以获取中断向量号。

2.2.2 存储器域地址空间到 PCI 总线域地址空间的转换

MPC8548 处理器使用 ATMU（Address Translation and Mapping Unit）寄存器组进行存储器域到 PCI 总线域，以及 PCI 总线域到存储器域的地址映射。ATMU 寄存器组由两大组寄存器组成，分别为 Outbound 和 Inbound 寄存器组。其中 Outbound 寄存器组将存储器域的地址转换为 PCI 总线域的地址，而 Inbound 寄存器组将 PCI 总线域的地址转换为存储器域的地址。

在 MPC8548 处理器中，只有当 CPU 读写访问的地址范围在 Outbound 寄存器组管理的地址空间之内时，HOST 主桥才能接收 CPU 的读写访问，并将 CPU 在存储器域上的读写访问转换为 PCI 总线域上的读写访问，然后才能对 PCI 设备进行读写操作。

如图 2-2 所示，CPU 对存储器域的地址访问，首先使用 CCB 总线事务，如果所访问的地址在 Cache 中命中，则从 Cache 中直接获得数据，否则将从存储器域中获取数据。而在绝大多数情况下，外部设备使用的地址空间是不可 Cache^①的，所以发向 PCI 设备的 CCB 总线

① PCI 设备使用的 ROM 空间可以是“可 Cache”的地址空间。

事务通常不会与 Cache 进行数据交换。

如果 CCB 总线事务使用的地址在 HOST 主桥的 Outbound 寄存器窗口中命中，HOST 主桥将接收这个 CCB 总线事务，并将其转换为 PCI 总线事务之后，再发送到 PCI 总线上。MPC8548 处理器的每一个 HOST 主桥都提供了 5 个 Outbound 寄存器窗口来实现存储器域地址到 PCI 总线域地址的映射，其映射过程如图 2-4 所示。

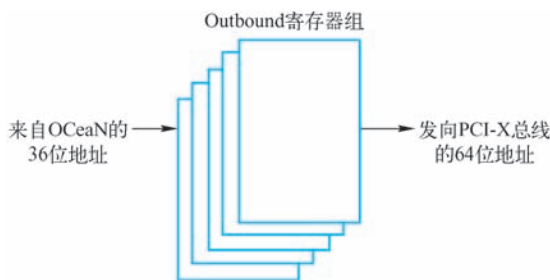


图 2-4 MPC8548 处理器存储器域到 PCI 总线域的转换

在介绍 MPC8548 处理器如何使用 Outbound 寄存器组进行存储器域地址空间到 PCI 总线域地址空间的转换之前，本节将首先介绍 Outbound 寄存器组中的相应寄存器。Outbound 寄存器组的地址偏移、属性和复位值如表 2-2 所示。

表 2-2 PCI/X ATMU Outbound 寄存器组

地 址 偏 移	寄 存 器	属 性	复 位 值
0x0_8C00/20/40/60/80	POTARn	可读写	0x0000-0000
0x0_8C04/24/44/64/84	POTEARn	可读写	0x0000-0000
0x0_8C28/48/68/88	POWBARn	可读写	0x0000-0000
0x0_8C30/50/70/90	POWARn	可读写	0x0000-0000

1. POTARn 和 POTEARn 寄存器

在 POTARn 和 POTEARn 寄存器中保存当前 Outbound 窗口在 PCI 总线域中的 64 位地址空间的基地址。这两个寄存器的主要字段如下。

- POTARn 寄存器的 TEA 字段，第 0~11 位，保存 PCI 总线地址空间的 43~32 位。
- POTARn 寄存器的 TA 字段，第 12~31 位，保存 PCI 总线地址空间的 31~12 位^①。
- POTEARn 寄存器的 TEA 字段，第 12~31 位，保存 PCI 总线地址空间的 63~44 位。

2. POWBARn 寄存器和 POWARn 寄存器

而 POWBARn 寄存器保存当前 Outbound 窗口在存储器域中的 36 位地址空间的基地址^②，其主要字段如下。

- WBEA 字段保存存储器域地址的第 0~3 位。
- WBA 字段保存存储器域地址的第 4~23^③位。

① POTARn 寄存器没有保存 PCI 总线的 11~0 位，因为 Outbound 窗口大小至少为 4 KB。

② MPC8548 处理器的物理地址为 36 位。注意在 PowerPC 处理器中，第 0 位是地址的最高位。

③ WBA 字段并没有保存存储器域的第 24~35 位地址，因为 Outbound 窗口大小至少为 4 KB 的整数倍。

POWARn 寄存器描述 Outbound 窗口的属性，其主要字段如下。

- EN 位，第 0 位。该位是 Outbound 窗口的使能位，为 1 表示当前 Outbound 寄存器组描述的存储器地址空间到 PCI 总线地址空间的映射关系有效；为 0 表示无效。
- RTT 字段，第 12~15 位，该字段描述当前窗口的读传送类型，为 0b0100 表示存储器读，为 0b1000 表示 I/O 读。
- WTT 字段，第 16~19 位，该字段描述当前窗口的写传送类型，为 0b0100 表示存储器写，为 0b1000 表示 I/O 写。在 PCIe 总线控制器中，RTT 字段和 WTT 字段还可以支持对配置空间的读写操作。
- OWS 字段，第 26~31 位，该字段描述当前窗口的大小，Outbound 窗口的大小在 4 KB~64 GB 之间，其值为 2^{OWS+1} 。

3. 使用 Outbound 寄存器访问 PCI 总线地址空间

MPC8548 处理器使用 Outbound 寄存器组访问 PCI 总线地址空间的步骤如下。

1) 首先 MPC8548 处理器需要将程序使用的 32 位有效地址 EA (Effective Address) 转换为 41 位的虚拟地址 VA (Virtual Address)。E500 V2 内核不能关闭 MMU (Memory Management Unit)，因此不能直接访问物理地址。

2) MPC8548 处理器通过 MMU 将 41 位的虚拟地址转换为 36 位的物理地址。在 E500 V2 内核中，物理地址是 36 位（默认是 32 位，需要使能）。

3) 检查 LAWBAR 和 LAWAR 寄存器，判断当前 36 位的物理地址是否属于 PCI 总线空间。在 MPC8548 中定义了一组 LAWBAR 和 LAWAR 寄存器对，每一对寄存器描述当前物理空间是与 PCI 总线、PCIe 总线、DDR 还是 RapidIO 空间对应。该组寄存器的详细说明见 MPC8548 PowerQUICC III™ Integrated Host Processor Family Reference Manual。如果 CPU 访问的空间为 PCI 总线空间，则执行第 4) 步，否则处理器将不会访问 PCI 地址空间。

4) 判断当前 36 位物理地址是否在 POWBARn 寄存器 1~4 描述的窗口中，如果在则将 36 位的处理器物理地址通过寄存器 POTARn 和 POTEARn 转换为 64 位的 PCI 总线地址，然后 HOST 主桥将来自处理器的读写请求发送到 PCI 总线上；如果不在 POWBARn 寄存器 1~4 描述的窗口中，POWBAR0 寄存器作为缺省窗口，接管这个存储器访问，并使用寄存器 POTAR0 和 POTEAR0，将处理器物理地址转换为 PCI 总线地址，当然在正常设计中很少出现这种情况。

许多系统软件，将 Outbound 窗口两边的寄存器使用“直接相等”的方法进行映射，将存储器域的地址与 PCI 总线地址设为相同的值。但是系统软件程序员务必注意这个存储器地址与 PCI 总线地址是分属于存储器域与 PCI 总线域的，这两个值虽然相等，但是所代表的地址并不相同，一个属于存储器域，而另一个属于 PCI 总线域。

2.2.3 PCI 总线域地址空间到存储器域地址空间的转换

MPC8548 处理器使用 Inbound 寄存器组将 PCI 总线域地址转换为存储器域的地址。PCI 设备进行 DMA 读写时，只有访问的地址在 Inbound 窗口中时，HOST 主桥才能接收这些读写请求，并将其转发到存储器控制器。MPC8548 处理器提供了 3 组 Inbound 寄存器，即提供 3 个 Inbound 寄存器窗口，实现 PCI 总线地址到存储器地址的反向映射。

从 PCI 设备的角度来看，PCI 设备访问存储器域的地址空间时，首先需要通过 Inbound 窗口将 PCI 总线地址转换为存储器域的地址；而从处理器的角度来看，处理器必须将存储器地址通过 Inbound 寄存器组反向映射为 PCI 总线地址空间，才能被 PCI 设备访问。

PCI 设备只能使用 PCI 总线地址访问 PCI 总线域的地址空间。HOST 主桥将这段地址空间通过 Inbound 窗口转换为存储器域的地址之后，PCI 设备才能访问存储器域地址空间。这个地址转换过程如图 2-5 所示。

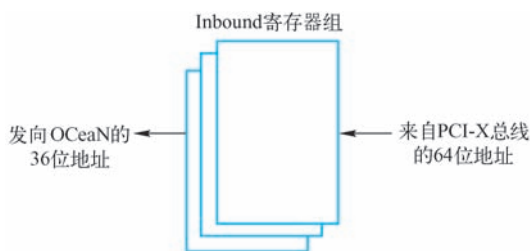


图 2-5 MPC8548 处理器 PCI 域到存储器域的转换

在介绍 MPC8548 处理器如何使用 Inbound 寄存器组进行 PCI 总线域地址空间到存储器域地址空间的转换之前，我们首先简要介绍 Inbound 寄存器组中的相应寄存器。该组寄存器的地址偏移、属性和复位值如表 2-3 所示。

表 2-3 PCI/X ATMU Inbound 寄存器组

地 址 偏 移	寄 存 器	属 性	复 位 值
0x0_8DA0/C0/E0	PITAR _n	可读写	0x0000-0000
0x0_8DA8/C8/E8	PIWBAR _n	可读写	0x0000-0000
0x0_8DAC/CC	PIWBEAR _n	可读写	0x0000-0000
0x0_8DB0/D0/F0	PIWAR _n	可读写	0x0000-0000

值得注意的是，Inbound 寄存器组除了可以进行 PCI 总线地址空间到存储器域地址空间的转换之外，还可以转换分属不同 PCI 总线域的地址空间，以支持 PCI 总线的 Peer-to-Peer 数据传送方式。

1. PITAR_n 寄存器

PITAR_n 寄存器保存当前 Inbound 窗口在存储器域中的 36 位地址空间的基地址，其地址窗口的大小至少为 4 KB，因此在该寄存器中仅存放存储器域地址的第 0~23 位，该寄存器的主要字段如下所示。

- TEA 字段存放存储器地址空间的第 0~3 位。
- TA 字段存放存储器地址空间的第 4~23 位。

2. PIWBAR_n 寄存器和 PIWBEAR_n 寄存器

PIWBAR_n 寄存器和 PIWBEAR_n 寄存器保存当前 Inbound 窗口在 PCI 总线域中的 64 位地址空间的基地址的第 63~12 位，Inbound 窗口使用的最小地址空间为 4 KB，因此在这两个寄存器中不含有 PCI 总线地址空间的第 11~0 位。这两个寄存器的主要字段如下所示。

- PIWBAR_n 寄存器的 BEA 字段存放 PCI 总线地址空间的第 43~32 位。

- PIWBARn 寄存器的 BA 字段存放 PCI 总线地址空间的第 31~12 位。
- PIWBEARn 寄存器的 BEA 字段存放 PCI 总线地址空间的第 63~44 位。

3. PIWARn 寄存器

PIWARn 寄存器描述当前 Inbound 窗口的属性，该寄存器由以下位和字段组成。

- EN 位，第 0 位。该位是 Inbound 窗口的使能位，为 1 表示当前 Inbound 寄存器组描述的存储器地址空间到 PCI 总线地址空间的映射关系有效；为 0 表示无效。
- PF 位，第 2 位。该位为 1 表示当前 Inbound 窗口描述的存储区域支持预读；为 0 表示不支持预读。
- TGI 字段，第 8~11 位。该字段为 0b0010 表示当前 Inbound 窗口描述的存储区域属于 PCIe 总线域地址空间；为 0b1100 表示当前 Inbound 窗口描述的存储区域属于 RapidIO 总线域地址空间。该字段对于 OCeAN 实现不同域间的报文转发非常重要，如果当前 Inbound 窗口的 TGI 字段为 0b0010，此时 PCI 总线上的设备可以使用该 Inbound 窗口，通过 OCeAN 直接读取 PCIe 总线的地址空间，而不需要经过 SoC 平台总线。如果 TGI 字段为 0b1111 表示 Inbound 窗口描述的存储器区域属于主存储器地址空间，这也是最常用的方式。使用该字段可以实现 HOST 主桥的 Peer-to-Peer 数据传送方式。
- RTT 字段和 WTT 字段，分别为该寄存器的第 12~15 位和第 16~19 位。Inbound 窗口的 RTT/WTT 字段的含义与 Outbound 窗口的 RTT/WTT 字段基本类似。只是在 Inbound 窗口中可以规定 PCI 设备访问主存储器时，是否需要进行 Cache 一致性操作（Cache Lock and Allocate），在进行 DMA 写操作时，数据是否可以直接进入 Cache。该字段是 PowerPC 处理器对 PCI 总线规范的有效补充，由于该字段的存在，PowerPC 处理器的 PCI 设备可以将数据直接写入 Cache，也可以视情况决定 DMA 操作是否需要进行 Cache 共享一致性操作。
- IWS 字段，第 26~31 位。该字段描述当前窗口的大小，Inbound 窗口的大小在 4 KB~16 GB 之间，其值为 2^{IWS+1} 。

4. 使用 Inbound 寄存器组进行 DMA 操作

PCI 设备使用 DMA 操作访问主存储器空间，或者访问其他 PCI 总线域地址空间时，需要通过 Inbound 窗口，其步骤如下。

1) PCI 设备在访问主存储器空间时，将首先检查当前 PCI 总线地址是否在 PIWBARn 和 PIWBEARn 寄存器描述的窗口中。如果在这个窗口中，则将这个 PCI 总线地址通过 PITARn 寄存器转换为存储器域的地址或者其他 PCI 总线域的地址；如果不在将禁止本次访问。

2) 如果 PCI 设备访问的是存储器地址空间，HOST 主桥将来自 PCI 总线的读写请求发送到存储器空间，进行存储器读写操作，并根据 Inbound 寄存器组的 RTT/WTT 位决定是否需要进行 Cache 一致性操作，或者将数据直接写入到 Cache 中。

结合 Outbound 寄存器组，可以发现 PCI 总线地址空间与存储器地址空间是有一定联系的。如果存储器域地址空间被 Inbound 寄存器组反向映射到 PCI 空间，这个存储器地址具有两个地址，一个是在存储器域的地址，一个是在 PCI 总线域的地址；同理 PCI 总线空间的地址如果使用 Outbound 寄存器映射到寄存器地址空间，这个 PCI 总线地址也具有两个地址，一个是在 PCI 总线域的地址，一个是在存储器域的地址。

能够被处理器和 PCI 总线同时访问的地址空间，一定在 PCI 总线域和存储器域中都存在地址映射。再次强调，绝大多数操作系统将同一个空间的 PCI 总线域地址和存储器地址设为相同的值，但是这两个相同的值所代表的含义不同。

由此可以看出，如果 MPC8548 处理器的某段存储器区域没有在 Inbound 窗口中定义时，PCI 设备将不能使用 DMA 机制访问这段存储器空间；同理如果 PCI 设备的空间不在 Outbound 窗口，HOST 处理器也不能访问这段 PCI 地址空间。

在绝大多数 PowerPC 处理器系统中，PCI 设备地址空间都在 HOST 主桥的 Outbound 窗口中建立了映射；而 MPC8548 处理器可以选择将哪些主存储器空间共享给 PCI 设备，从而对主存储器空间进行保护。

2.2.4 x86 处理器的 HOST 主桥

x86 处理器使用南北桥结构连接 CPU 和 PCI 设备。其中北桥（North Bridge）连接快速设备，如显卡和内存条，并推出 PCI 总线，HOST 主桥包含在北桥中。而南桥（South Bridge）连接慢速设备。x86 处理器使用的南北桥结构如图 2-6 所示。

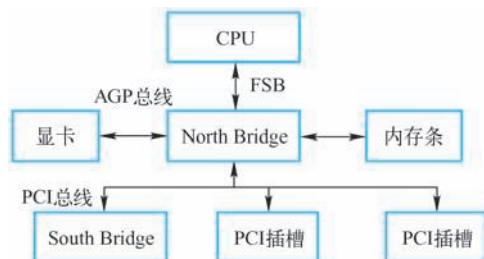


图 2-6 x86 处理器的南北桥结构

Intel 使用南北桥概念统一 PC 架构。但是从体系结构的角度上看，南北桥架构并不重要，北桥中存放的主要部件不过是存储器控制器、显卡控制器和 HOST 主桥而已，而南桥存放的是一些慢速设备，如 ISA 总线和中断控制器等。

不同的处理器系统集成这些组成部件的方式并不相同，如 PowerPC、MIPS 和 ARM 处理器系统通常将 CPU 和主要外部设备都集成到一颗芯片中，组成一颗基于 SoC 架构的处理器系统。这些集成方式并不重要，每一个处理器系统都有其针对的应用领域，不同应用领域的需求对处理器系统的集成方式有较大的影响。Intel 采用的南北桥架构针对 x86 处理器的应用领域而设计，并不能说采用这种结构一定比 MPC8548 处理器中既含有 HOST-to-PCI 主桥也含有 HOST-to-PCIe 主桥更为合理。

在许多嵌入式处理器系统中，既含有 PCI 设备也含有 PCIe 设备，为此 MPC8548 处理器同时提供了 PCI 总线和 PCIe 总线接口，在这个处理器系统中，PCI 设备可以与 PCI 总线直接相连，而 PCIe 设备可以与 PCIe 总线直接相连，因此并不需要使用 PCIe 桥扩展 PCI 总线，从而在一定程度上简化了嵌入式系统的设计。

嵌入式系统所面对的应用千变万化，进行芯片设计时所要考虑的因素相对较多，因而在某种程度上为设计带来了一些难度。而 x86 处理器系统所面对的应用领域针对个人 PC 和服务

过专用处理器系统的设计，Intel 为此付出了极大的代价。

在一些相对较老的北桥中，如 Intel 440 系列芯片组中包含了 HOST 主桥，从系统软件的角度上看 HOST-to-PCI 主桥实现的功能与 HOST-to-PCIe 主桥实现的功能相近。本节仅简单介绍 Intel 的 HOST-to-PCI 主桥如何产生 PCI 的配置周期，有关 Intel HOST-to-PCIe 主桥^①的详细信息参见第 5 章。

x86 处理器定义了两个 I/O 端口寄存器，分别为 CONFIG_ADDRESS 和 CONFIG_DATA 寄存器，其地址为 0xCF8 和 0xCFC。x86 处理器使用这两个 I/O 端口访问 PCI 设备的配置空间。PCI 总线规范也以这两个寄存器为例，说明处理器如何访问 PCI 设备的配置空间。其中 CONFIG_ADDRESS 寄存器存放 PCI 设备的 ID 号，而 CONFIG_DATA 寄存器存放进行配置读写的数据。

CONFIG_ADDRESS 寄存器与 PowerPC 处理器中的 CFG_ADDR 寄存器的使用方法类似，而 CONFIG_DATA 寄存器与 PowerPC 处理器中的 CFG_DATA 寄存器的使用方法类似。CONFIG_ADDRESS 寄存器的结构如图 2-7 所示。

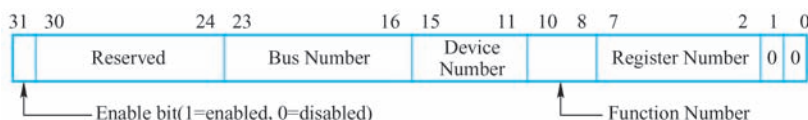


图 2-7 CONFIG_ADDRESS 寄存器的结构

CONFIG_ADDRESS 寄存器的各个字段和位的说明如下所示。

- Enable 位，第 31 位。该位为 1 时，对 CONFIG_DATA 寄存器进行读写时将引发 PCI 总线的配置周期。
- Bus Number 字段，第 23~16 位，记录 PCI 设备的总线号。
- Device Number 字段，第 15~11 位，记录 PCI 设备的设备号。
- Function Number 字段，第 10~8 位，记录 PCI 设备的功能号。
- Register Number 字段，第 7~2 位，记录 PCI 设备的寄存器号。

当 x86 处理器对 CONFIG_DATA 寄存器进行 I/O 读写访问，且 CONFIG_ADDR 寄存器的 Enable 位为 1 时，HOST 主桥将这个 I/O 读写访问转换为 PCI 配置读写总线事务，然后发送到 PCI 总线上，PCI 总线根据保存在 CONFIG_ADDR 寄存器中的 ID 号，将 PCI 配置读写请求发送到指定 PCI 设备的指定配置寄存器中。

x86 处理器使用小端地址模式，因此从 CONFIG_DATA 寄存器中读出的数据不需要进行模式转换，这点和 PowerPC 处理器不同，此外 x86 处理器的 HOST 主桥也实现了存储器域到 PCI 总线域的地址转换，但是这个概念在 x86 处理器中并不明晰。

本书将在第 5 章以 HOST-to-PCIe 主桥为例，详细介绍 Intel 处理器的存储器地址与 PCI 总线地址的转换关系，而在本节不对 x86 处理器的 HOST 主桥做进一步说明。x86 处理器系统的升级速度较快，目前在 x86 的处理器体系结构中，已很难发现 HOST 主桥的身影。

目前 Intel 对南北桥架构进行了升级，其中北桥被升级为 MCH (Memory Controller Hub)，

① 这个 HOST-to-PCIe 主桥也是 RC (Root Complex) 的一部分。

而南桥被升级为 ICH (I/O Controller Hub)。x86 处理器系统在 MCH 中集成了存储器控制器、显卡芯片和 HOST-to-PCIe 主桥, 并通过 Hub Link 与 ICH 相连; 而在 ICH 中集成了一些相对低速总线接口, 如 AC'97、LPC (Low Pin Count)、IDE 和 USB 总线, 当然也包括一些低带宽的 PCIe 总线接口。

在 Intel 最新的 Nehalem^① 处理器系统中, MCH 被一分为二, 存储器控制器和图形控制器已经与 CPU 内核集成在一个 DIE 中, 而 MCH 剩余的部分与 ICH 合并成为 PCH (Peripheral Controller Hub)。但是从体系结构的角度上看, 这些升级与整合并不重要。

目前 Intel 在 Menlow^② 平台基础上, 计划推出基于 SoC 架构的 x86 处理器, 以进军手持设备市场。在基于 SoC 构架的 x86 处理器中将逐渐淡化 Chipset 的概念, 其拓扑结构与典型的 SoC 处理器, 如 ARM 和 PowerPC 处理器, 较为类似。

2.3 PCI 桥与 PCI 设备的配置空间

PCI 设备都有独立的配置空间, HOST 主桥通过配置读写总线事务访问这段空间。PCI 总线规定了三种类型的 PCI 配置空间, 分别是 PCI Agent 设备使用的配置空间, PCI 桥使用的配置空间和 Cardbus 桥片使用的配置空间。

本节重点介绍 PCI Agent 和 PCI 桥使用的配置空间, 而并不介绍 Cardbus 桥片使用的配置空间。值得注意的是, 在 PCI 设备配置空间中出现的地址都是 PCI 总线地址, 属于 PCI 总线域地址空间。

2.3.1 PCI 桥

PCI 桥的引入使 PCI 总线极具扩展性, 也极大地增加了 PCI 总线的复杂度。PCI 总线的电气特性决定了在一条 PCI 总线上挂接的负载有限, 当 PCI 总线需要连接多个 PCI 设备时, 需要使用 PCI 桥进行总线扩展, 扩展出的 PCI 总线可以连接其他 PCI 设备, 包括 PCI 桥。在一棵 PCI 总线树上, 最多可以挂接 256 个 PCI 设备, 包括 PCI 桥。PCI 桥在 PCI 总线树中的位置如图 2-8 所示。

PCI 桥作为一个特殊的 PCI 设备, 具有独立的配置空间。但是 PCI 桥配置空间的定义与 PCI Agent 设备有所不同。PCI 桥的配置空间可以管理其下 PCI 总线子树的 PCI 设备, 并可以优化这些 PCI 设备通过 PCI 桥的数据访问。PCI 桥的配置空间在系统软件遍历 PCI 总线树时进行配置, 系统软件不需要专门的驱动程序设置 PCI 桥的使用方法, 这也是 PCI 桥被称为透明桥的主要原因。

在某些处理器系统中, 还有一类 PCI 桥, 叫作非透明桥。非透明桥不是 PCI 总线定义的标准桥片, 但是在使用 PCI 总线挂接另外一个处理器系统时非常有用, 非透明桥片的主要作用是连接两个不同的 PCI 总线域, 进而连接两个处理器系统, 本章将在第 2.5 节中详细介绍 PCI 非透明桥。

① Nehalem 处理器也称为 Core i7 处理器。

② Menlow 平台于 2008 年 3 月发布, 其目标应用为 MID (Mobile Internet Device) 设备。Menlow 平台基于低功耗处理器内核 Atom。

使用 PCI 桥可以扩展出新的 PCI 总线，在这条 PCI 总线上还可以继续挂接多个 PCI 设备。PCI 桥跨接在两个 PCI 总线之间，其中距离 HOST 主桥较近的 PCI 总线被称为该桥片的上游总线（Primary Bus），距离 HOST 主桥较远的 PCI 总线被称为该桥片的下游总线（Secondary Bus）。如图 2-8 所示，PCI 桥 1 的上游总线为 PCI 总线 x0，而 PCI 桥 1 的下游总线为 PCI 总线 x1。这两条总线间的数据通信需要通过 PCI 桥 1。

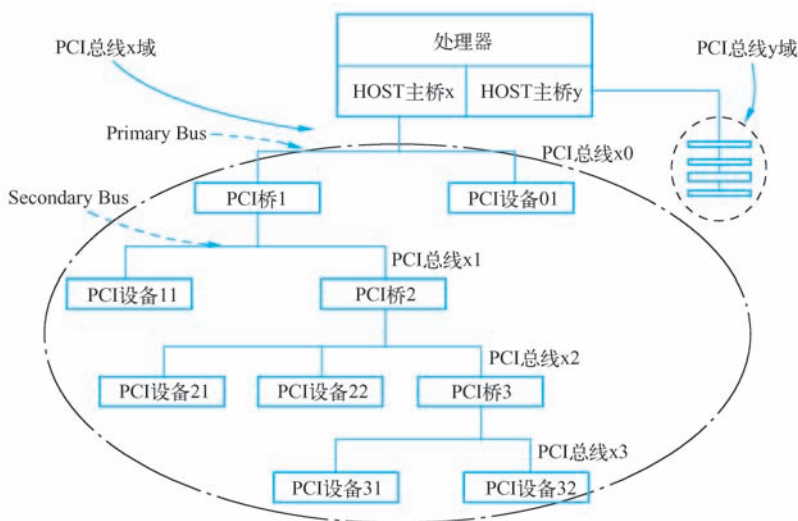


图 2-8 使用 PCI 桥扩展 PCI 总线

通过 PCI 桥连接的 PCI 总线属于同一个 PCI 总线域，在图 2-8 中，PCI 桥 1、2 和 3 连接的 PCI 总线都属于 PCI 总线 x 域。在这些 PCI 总线域上的设备可以通过 PCI 桥直接进行数据交换而不需要进行地址转换；而分属不同 PCI 总线域的设备间的通信需要进行地址转换，如与 PCI 非透明桥两端连接的设备之间的通信。

如图 2-8 所示，每一个 PCI 总线的下方都可以挂接一个到多个 PCI 桥，每一个 PCI 桥都可以推出一条新的 PCI 总线。在同一条 PCI 总线上的设备之间的数据交换不会影响其他 PCI 总线。如 PCI 设备 21 与 PCI 设备 22 之间的数据通信仅占用 PCI 总线 x2 的带宽，而不会影响 PCI 总线 x0、x1 与 x3，这也是引入 PCI 桥的一个重要原因。

由图 2-8 还可以发现 PCI 总线可以通过 PCI 桥组成一个胖树结构，其中每一个桥片都是父节点，而 PCI Agent 设备只能是子节点。当 PCI 桥出现故障时，其下的设备不能将数据传递给上游总线，但是并不影响 PCI 桥下游设备间的通信。当 PCI 桥 1 出现故障时，PCI 设备 11、PCI 设备 21 和 PCI 设备 22 将不能与 PCI 设备 01 和存储器进行通信，但是 PCI 设备 21 和 PCI 设备 22 之间的通信可以正常进行。

使用 PCI 桥可以扩展一条新的 PCI 总线，但是不能扩展新的 PCI 总线域。如果当前系统使用 32 位的 PCI 总线地址，那么这个系统的 PCI 总线域的地址空间为 4 GB 大小，在这个总线域上的所有设备将共享这个 4GB 大小的空间。如在 PCI 总线 x 域上的 PCI 桥 1、PCI 设备 01、PCI 设备 11、PCI 桥 2、PCI 设备 21 和 PCI 设备 22 等都将共享一个 4 GB 大小的空间。再次强调这个 4 GB 空间是 PCI 总线 x 域的“PCI 总线地址空间”，和存储器域地址空间和 PCI 总线 y 域没有直接联系。

处理器系统可以通过 HOST 主桥扩展出新的 PCI 总线域，如 MPC8548 处理器的 HOST 主桥 x 和 y 可以扩展出两个 PCI 总线域 x 和 y。这两个 PCI 总线域 x 和 y 之间的 PCI 空间在正常情况下不能直接进行数据交换，但是 PowerPC 处理器可以通过设置 PIWARn 寄存器的 TGI 字段使得不同 PCI 总线域的设备直接通信，详见第 2.2.3 节。

许多处理器系统使用的 PCI 设备较少，因而并不需要使用 PCI 桥。因此在这些处理器系统中，PCI 设备都是直接挂接在 HOST 主桥上，而不需要使用 PCI 桥扩展新的 PCI 总线。即便如此读者也需要深入理解 PCI 桥的知识。

PCI 桥对于理解 PCI 和 PCIe 总线都非常重要。在 PCIe 总线中，虽然在物理结构上并不含有 PCI 桥，但是与 PCI 桥相关的知识在 PCIe 总线中无处不在，比如在 PCIe 总线的 Switch 中，每一个端口都与一个虚拟 PCI 桥对应，Switch 使用这个虚拟 PCI 桥管理其下 PCI 总线子树的地址空间。

2.3.2 PCI Agent 设备的配置空间

在一个具体的处理器应用中，PCI 设备通常将 PCI 配置信息存放在 E²PROM 中。PCI 设备进行上电初始化时，将 E²PROM 中的信息读到 PCI 设备的配置空间中作为初始值。这个过程由硬件逻辑完成，绝大多数 PCI 设备使用这种方式初始化其配置空间。

读者可能会对这种机制产生一个疑问，如果系统软件在 PCI 设备将 E²PROM 中的信息读到配置空间之前，就开始操作配置空间，会不会带来问题？因为此时 PCI 设备的初始值并不“正确”，仅仅是 PCI 设备使用的复位值。

读者的这种担心是多余的，因为 PCI 设备在配置寄存器没有初始化完毕之前，即 E²PROM 中的内容没有导入 PCI 设备的配置空间之前，可以使用 PCI 总线规定的“Retry”周期使 HOST 主桥在合适的时机重新发起配置读写请求。

在 x86 处理器中，系统软件使用 CONFIG_ADDR 和 CONFIG_DATA 寄存器，读取 PCI 设备配置空间的这些初始化信息，然后根据处理器系统的实际情况使用 DFS 算法，初始化处理器系统中所有 PCI 设备的配置空间。

在 PCI Agent 设备的配置空间中包含了许多寄存器，这些寄存器决定了该设备在 PCI 总线中的使用方法，本节不会全部介绍这些寄存器，因为系统软件只对部分配置寄存器感兴趣。PCI Agent 设备使用的配置空间如图 2-9 所示。

在 PCI Agent 设备配置空间中包含的寄存器如下所示。

(1) Device ID 和 Vendor ID 寄存器

这两个寄存器的值由 PCISIG 分配，只读。其中 Vendor ID 代表 PCI 设备的生产厂商，而 Device ID 代表这个厂商所生产的具体设备。如 Intel 公司的基于 82571EB 芯片的系列网卡，其 Vendor ID 为 0x8086[Ⓐ]，而 Device ID 为 0x105E[Ⓑ]。

其中 0x8086 代表 Intel，0x105E 代表 82571EB 网卡芯片。Intel 将 0x10xx 作为 LAN 设备的 Device ID。Intel 在 PCISIG 上注册了多如牛毛的 Device ID，这些 Device ID 放在一起，几页纸也列不完。不过 16 位的 Device ID 即便对于 Intel 这样大的公司，也基本没有用完的可能。当 Vendor ID 寄存器为 0xFFFF 时，表示为无效 Vendor ID。

Ⓐ PCI SIG 分配给 Intel 的 Vendor ID 号是 0x8086，8086 处理器也是 Intel 设计的第一个 PC 处理器。

Ⓑ 这仅是 Intel 为 82571 的 Copper 口分配的 Vendor ID。

31	24	23	16	15	8	7	0	
Device ID				Vendor ID				0x00
Status				Command				0x04
Class Code						Revision ID		0x08
BIST		Header Type		Latency Timer		Cache Line Size		0x0C
Base Address Register0								0x10
Base Address Register1								0x14
Base Address Register2								0x18
Base Address Register3								0x1C
Base Address Register4								0x20
Base Address Register5								0x24
Cardbus CIS Pointer								0x28
Subsystem ID				Subsystem Vendor ID				0x2C
Expansion ROM Base Address								0x30
Reserved						Capabilities Pointer		0x34
Reserved								0x38V
MAX_Lat		Min_Gnt		Interrupt Pin		Interrupt Line		0x3C

图 2-9 PCI Agent 设备的配置空间

(2) Revision ID 和 Class Code 寄存器

这两个寄存器只读。其中 Revision ID 寄存器记载 PCI 设备的版本号。该寄存器可以被认为是 Device ID 寄存器的扩展。

而 Class Code 寄存器记载 PCI 设备的分类，该寄存器由三个字段组成，分别是 Base Class Code、Sub Class Code 和 Interface。其中 Base Class Code 将 PCI 设备分类为显卡、网卡、PCI 桥等设备；Sub Class Code 对这些设备进一步细分；而 Interface 定义编程接口。Class Code 寄存器可供系统软件识别当前 PCI 设备的分类。

除此之外硬件逻辑设计也需要使用该寄存器识别不同的设备。当 Base Class Code 寄存器为 0x06，Sub Class Code 寄存器为 0x04 时，如果 Interface 寄存器为 0x00，表示当前 PCI 设备为一个标准的 PCI 桥；如果 Interface 寄存器为 0x01，表示当前 PCI 设备为一个使用“负向译码”的 PCI 桥。

硬件逻辑需要根据这些寄存器判断当前 PCI 桥的使用方式，许多 PCI 桥既可以支持“正向”译码，也可以支持“负向”译码，系统软件必须合理设置 Class Code 寄存器。有关正向译码与负向译码的详细说明见第 3.2.1 节。

(3) Header Type 寄存器

该寄存器只读，由 8 位组成。

- 第 7 位为 1 表示当前 PCI 设备是多功能设备，为 0 表示为单功能设备。
- 第 6~0 位表示当前配置空间的类型，为 0 表示该设备使用 PCI Agent 设备的配置空间，普通 PCI 设备都使用这种配置头；为 1 表示使用 PCI 桥的配置空间，PCI 桥使用这种配置头；为 2 表示使用 Cardbus 桥片的配置空间，Card Bus 桥片使用这种配置头，本书对这类配置头不作详解。

系统软件需要使用该寄存器区分不同类型的 PCI 配置空间，该寄存器的初始化必须与 PCI 设备的实际情况对应，而且必须为一个合法值。

(4) Cache Line Size 寄存器

该寄存器记录 HOST 处理器使用的 Cache 行长度。在 PCI 总线中和 Cache 相关的总线事务，如存储器写并无效和 Cache 多行读等总线事务需要使用这个寄存器。值得注意的是，该寄存器由系统软件设置，但是在 PCI 设备的运行过程中，只有其硬件逻辑才会使用该寄存器，比如 PCI 设备的硬件逻辑需要得知处理器系统 Cache 行的大小，才能进行存储器写并无效总线事务，单行读和多行读总线事务。

如果 PCI 设备不支持与 Cache 相关的总线事务，系统软件可以不设置该寄存器，此时该寄存器为初始值 0x00。对于 PCIe 设备，该寄存器的值无意义，因为 PCIe 设备在进行数据传送时，在其报文中含有一次数据传送的大小，PCIe 总线控制器可以使用这个“大小”，判断数据区域与 Cache 行的对应关系。

(5) Subsystem ID 和 Subsystem Vendor ID 寄存器

这两个寄存器和 Device ID 及 Vendor ID 类似，也是记录 PCI 设备的生产厂商和设备名称。但是这两个寄存器和 Device ID 及 Vendor ID 寄存器略有不同。下面以一个实例说明 Subsystem ID 和 Subsystem Vendor ID 的用途。

Xilinx 公司在 FPGA 中集成了一个 PCIe 总线接口的 IP 核，即 LogiCORE。用户可以使用 LogiCORE 设计各种各样基于 PCIe 总线的设备，但是这些设备的 Device ID 都是 0x10EE，而 Vendor ID 为 0x0007^①。

因此仅使用 Device ID 和 Vendor ID 寄存器无法区分这些设备。此时必须使用 Subsystem ID 和 Subsystem Vendor ID。如果 Intel 也使用 LogiCORE 设计一款网卡适配器，那么这个基于 LogiCORE 的网卡适配器的 Subsystem Vendor ID 寄存器为 0x8086，而 Subsystem ID 寄存器将是 0x10xx。

(6) Expansion ROM Base Address 寄存器

有些 PCI 设备在处理器还没有运行操作系统之前，就需要完成基本的初始化设置，比如显卡、键盘和硬盘等设备。为了实现这个“预先执行”功能，PCI 设备需要提供一段 ROM 程序，而处理器在初始化过程中将运行这段 ROM 程序，初始化这些 PCI 设备。Expansion ROM base address 记载这段 ROM 程序的基地址。

(7) Capabilities Pointer 寄存器

在 PCI 设备中，该寄存器是可选的，但是在 PCI-X 和 PCIe 设备中必须支持这个寄存器，Capabilities Pointer 寄存器存放 Capabilities 寄存器组的基地址，PCI 设备使用 Capabilities 寄存器组存放一些与 PCI 设备相关的扩展配置信息。该组寄存器的详细说明见第 4.3 节。

(8) Interrupt Line 寄存器

这个寄存器是系统软件对 PCI 设备进行配置时写入的，该寄存器记录当前 PCI 设备使用的中断向量号，设备驱动程序可以通过这个寄存器，判断当前 PCI 设备使用处理器系统中的哪个中断向量号，并将驱动程序的中断服务例程注册到操作系统中^②。

该寄存器由系统软件初始化，其保存的值与 8259A 中断控制器相关，该寄存器的值也是由 PCI 设备与 8259A 中断控制器的连接关系决定的。如果在一个处理器系统中，没有使

① Xilinx 使用的 Device ID 号为 0x10EE，而 LogiCORE 的 Vendor ID 号为 0x0007。

② Linux 系统使用 request_irq 函数注册一个设备的中断服务例程。

用 8259A 中断控制器管理 PCI 设备的中断，则该寄存器中的数据并没有意义。

在多数 PowerPC 处理器系统中，并不使用 8259A 中断控制器管理 PCI 设备的中断请求，因此该寄存器没有意义。即使在 x86 处理器系统中，如果使用 I/O APIC 中断控制器，该寄存器保存的内容仍然无效。目前在绝大多数处理器系统中，并没有使用该寄存器存放 PCI 设备使用的中断向量号。

(9) Interrupt Pin 寄存器

这个寄存器保存 PCI 设备使用的中断引脚。PCI 总线提供了四个中断引脚：INTA#、INTB#、INTC#和 INTD#。Interrupt Pin 寄存器为 1 时表示使用 INTA#引脚向中断控制器提交中断请求，为 2 表示使用 INTB#，为 3 表示使用 INTC#，为 4 表示使用 INTD#。

如果 PCI 设备只有一个子设备时，该设备只能使用 INTA#；如果有多个子设备时，可以使用 INTB~D#信号。如果 PCI 设备不使用这些中断引脚，向处理器提交中断请求时，该寄存器的值必须为 0。值得注意的是，虽然在 PCIe 设备中并不含有 INTA~D#信号，但是依然可以使用该寄存器，因为 PCIe 设备可以使用 INTx 中断消息，模拟 PCI 设备的 INTA~D#信号，详见第 6.3.4 节。

(10) Base Address Register 0~5 寄存器

该组寄存器简称为 BAR 寄存器，BAR 寄存器保存 PCI 设备使用的地址空间的基地址，该基地址保存的是该设备在 PCI 总线域中的地址。其中每一个设备最多可以有 6 个基址空间，但多数设备不会使用这么多组地址空间。

在 PCI 设备复位之后，该寄存器将存放 PCI 设备需要使用的基址空间大小，这段空间是 I/O 空间还是存储器空间[⊖]，如果是存储器空间该空间是否可预取，有关 PCI 总线预读机制的详细说明见第 3.4.5 节。

系统软件对 PCI 总线进行配置时，首先获得 BAR 寄存器中的初始化信息，之后根据处理器系统的配置，将合理的基地址写入相应的 BAR 寄存器中。系统软件还可以使用该寄存器，获得 PCI 设备使用的 BAR 空间的长度，其方法是向 BAR 寄存器写入 0xFFFF-FFFF，之后再读取该寄存器。Linux 系统使用 pci_read_base 函数获得 BAR 寄存器的长度，其步骤详见第 14.3.2 节。

处理器访问 PCI 设备的 BAR 空间时，需要使用 BAR 寄存器提供的基地址。值得注意的是，处理器使用存储器域的地址，而 BAR 寄存器存放 PCI 总线域的地址。因此处理器系统并不能直接使用“BAR 寄存器+偏移”的方式访问 PCI 设备的寄存器空间，而需要将 PCI 总线域的地址转换为存储器域的地址。在 Linux 系统中，一个处理器系统使用 BAR 空间的正确方式如源代码 2-2 所示。

源代码 2-2 Linux 系统使用 BAR 空间的正确方法

```
pciaddr = pci_resource_start(pdev,1);

if (! pciaddr) {
    rc = -EIO;
    dev_err(&pdev->dev,"no MMIO resource\n");
```

⊖ 一般来说 PCI 设备使用 E²PROM 保存 BAR 寄存器的初始值。

```

        goto err_out_res;
    }
    ...
    regs = ioremap(pciaddr, CP_REGS_SIZE);

```

在 Linux 系统中，使用 `pci_dev→resource[bar].start` 参数保存 BAR 寄存器在存储器域的地址。在编写 Linux 设备驱动程序时，必须使用 `pci_resource_start` 函数获得 BAR 空间对应的存储器域的物理地址，而不能使用从 BAR 寄存器中读出的地址。

当驱动程序获得 BAR 空间在存储器域的物理地址后，再使用 `ioremap` 函数将这个物理地址转换为虚拟地址。Linux 系统直接使用 BAR 空间的方法是不正确的，如源代码 2-3 所示。

源代码 2-3 Linux 系统使用 BAR 空间的错误方法

```

    ret = pci_read_config_dword(pdev, 1, &pciaddr);

    if (! pciaddr) {
        rc = -EIO;
        dev_err(&pdev->dev, "no MMIO resource\n");
        goto err_out_res;
    }
    ...
    regs = ioremap(pciaddr, BAR_SIZE);

```

在 Linux 系统中，使用 `pci_read_config_dword` 函数获得的是 PCI 总线域的物理地址，在许多处理器系统中，如 Alpha 和 PowerPC 处理器系统，PCI 总线域的物理地址与存储器域的物理地址并不相等。

如果 x86 处理器系统使能了 IOMMU 后，这两个地址也并不一定相等，因此处理器系统直接使用这个 PCI 总线域的物理地址，并不能确保访问 PCI 设备的 BAR 空间的正确性。除此之外在 Linux 系统中，`ioremap` 函数的输入参数为存储器域的物理地址，而不能使用 PCI 总线域的物理地址。

而在 `pci_dev→resource[bar].start` 参数中保存的地址已经经过 PCI 总线域到存储器域的地址转换，因此在编写 Linux 系统的设备驱动程序时，需要使用 `pci_dev→resource[bar].start` 参数中的物理地址，再用 `ioremap` 函数将物理地址转换为“存储器域”的虚拟地址。

(11) Command 寄存器

该寄存器为 PCI 设备的命令寄存器，在初始化时，其值为 0，此时这个 PCI 设备除了能够接收配置请求总线事务之外，不能接收任何存储器或者 I/O 请求。系统软件需要合理设置该寄存器之后，才能访问该设备的存储器或者 I/O 空间。在 Linux 系统中，设备驱动程序调用 `pci_enable_device` 函数[⊖]，使能该寄存器的 I/O 和 Memory Space 位之后，才能访问该设备的存储器或者 I/O 地址空间。Command 寄存器的各位的含义如表 2-4 所示。

⊖ `pci_enable_device` 函数的详细说明第 12.3.2 节。

表 2-4 Command 寄存器

位	描 述
0	I/O Space 位, 该位表示 PCI 设备是否响应 I/O 请求, 为 1 时响应, 为 0 时不响应。如果 PCI 设备支持 I/O 地址空间, 系统软件会将该位置 1。复位值为 0
1	Memory Space 位, 该位表示 PCI 设备是否响应存储器请求, 为 1 时响应, 为 0 时不响应。如果 PCI 设备支持存储器地址空间, 系统软件会将该位置 1。复位值为 0
2	Bus Master 位。该位表示 PCI 设备是否可以作为主设备, 为 1 时 PCI 设备可以作为主设备, 为 0 时不能。复位值为 0
3	Special Cycle 位, 该位表示 PCI 设备是否响应 Special 总线事务, 为 1 时响应, 为 0 时不响应。PCI 设备可以使用 Special 总线事务, 将一些信息广播发送到多个目标设备, Special 总线事务不能穿越 PCI 桥。如果一个 PCI 设备需要将 Special 总线事务发送到 PCI 桥之下的总线时, 必须使用 Type 01h 配置周期。PCI 桥可以将 Type 01h 配置周期转换为 Special 周期。该位的复位值为 0
4	Memory Write and Invalidate 位, 该位表示 PCI 设备是否支持 Memory Write and Invalidate 总线事务, 为 1 时支持, 为 0 时不支持。许多低端的 PCI 设备不支持这种总线事务。该位对 PCIe 设备无意义
5	VGA Palette Snoop 位。该位为 1 时支持 Palette Snoop 功能, 为 0 时不支持
6	Parity Error Response 位, 复位值为 0。该位为 1, 而且 PCI 设备在传送过程中出现奇偶校验错误时, PCI 设备将 PERR#信号设置为 1; 该位为 0 时, 即便出现奇偶检验错误, PCI 设备也仅会将 Status 寄存器的“Detected Parity Error”位置 1
8	SERR# Enable 位, 复位值为 0。该位为 1, 而且 PCI 设备出现错误时, 将使用 SERR#信号, 将这个错误信息发送给 HOST 主桥, 为 0 时, 不能使用 SERR#信号
9	Fast Back-to-Back 位。该位为 1 时, PCI 设备使用 Fast Back-to-Back (快速背靠背) 总线周期, 这种周期是一种提高传送效率的方法。但并不是所有的 PCI 设备都支持 Fast Back-to-Back 传送周期。该位的复位值为 0
10	Interrupt Disable 位, 复位值为 0。该位为 1 时, PCI 设备不能通过 INTx 信号向 HOST 主桥提交中断请求, 为 0 时可以使用 INTx 信号提交中断请求。当 PCI 设备使用 MSI 中断方式提交中断请求时, 该位将被置为 1

(12) Status 寄存器

该寄存器的绝大多数位都是只读位, 保存 PCI 设备的状态, 其含义如表 2-5 所示。

表 2-5 Status 寄存器

位	描 述
3	Interrupt Status 位, 该位只读。该位为 1 且 Command 寄存器的 Interrupt Disable 位为 0 时, 表示 PCI 设备已经使用 INTx 信号向处理器提交了中断请求。在多数 PCI 设备中的 BAR 空间, 存在自定义的中断状态寄存器, 因此设备驱动程序很少使用该位判断 PCI 设备是否提交了中断请求
4	Capabilities List 位, 该位只读。该位为 1 时 Capability Pointer 寄存器中的值有效。本书在第 4.3 节详细介绍 PCI 设备的 Capability Pointer 寄存器
5	66MHz Capability 位, 该位只读。为 1 时表示此设备支持 66 MHz 的 PCI 总线
7	Fast Back-to-Back Capable 位。该位只读, 该位为 1 表示此设备支持快速“背靠背”总线周期
8	Master Data Parity Error 位。PCI 总线的 PERR#信号有效时将置该位为 1; 当 PCI 总线出现数据传送错误时置此位为 1; 当 Command 寄存器的 Parity Error Response 位为 1 时, 此位为 1
9~10	DEVSEL timing 字段。该字段为 0b00 时表示 PCI 设备为快速设备; 为 0b01 时表示 PCI 设备为中速设备; 为 0b10 时表示 PCI 设备为慢速设备。快速设备要求 PCI 总线主设备置 FRAME#信号有效的一个时钟周期后, 置 DEVSEL#信号有效; 中速设备要求 PCI 总线主设备置 FRAME#信号有效的两个时钟周期后, 置 DEVSEL#信号有效; 慢速设备要求 PCI 总线主设备置 FRAME#信号有效的三个时钟周期后, 置 DEVSEL#信号有效

(续)

位	描 述
9~10	在一条 PCI 总线上, 如果快速设备、中速设备和慢速设备都没有使用 DEVSEL#信号响应当前总线事务时, 这条总线上的负向译码设备, 将被动地接收这个总线事务。如果在这条总线上没有负向译码设备, 主设备在 FRAME#信号有效后的第 4 个时钟周期, 使用主设备夭折时序, 结束当前总线事务。有关负向译码设备的详细说明见第 3.2.1 节。 值得注意的是, 在 PCI-X 总线中, 该字段的含义与 PCI 总线有所不同
11	Signaled Target Abort 位。该位由 PCI 目标设备设置, 当目标设备使用目标设备夭折 (Target Abort) 时序, 结束当前总线周期时, PCI 设备将置该位为 1
12	Received Target Abort 位。该位由 PCI 主设备设置, 当发生目标设备夭折时序时, 该位被置为 1
13	Received Master Abort 位。该位由 PCI 主设备设置, 当发生主设备夭折时序, 该位被置为 1。当以上几个 Abort 位有效时, 表示 PCI 总线的数据传送通路出现了较为严重的问题
14	Signaled System Error 位。当设备置 SERR#信号有效时, 该位被置 1
15	Detected Parity Error 位。当设备发现奇偶校验错时, 该位被置 1

(13) Latency Timer 寄存器

在 PCI 总线中, 多个设备共享同一条总线带宽。该寄存器用来控制 PCI 设备占用 PCI 总线的时间, 当 PCI 设备获得总线使用权, 并使能 Frame#信号后, Latency Timer 寄存器将递减, 当该寄存器归零后, 该设备将使用超时机制停止[⊖]对当前总线的使用。

如果当前总线事务为 Memory Write and Invalidate 时, 需要保证对一个完整 Cache 行的操作结束后才能停止当前总线事务。对于多数 PCI 设备而言, 该寄存器的值为 32 或者 64, 以保证一次突发传送的基本单位为一个 Cache 行。

PCIe 设备不需要使用该寄存器, 该寄存器的值必须为 0。因为 PCIe 总线的仲裁方法与 PCI 总线不同, 使用的连接方法也与 PCI 总线不同。

2.3.3 PCI 桥的配置空间

PCI 桥使用的配置空间的寄存器如图 2-10 所示。PCI 桥作为一个 PCI 设备, 使用的许多配置寄存器与 PCI Agent 的寄存器是类似的, 如 Device ID、Vendor ID、Status、Command、Interrupt Pin、Interrupt Line 寄存器等, 本节不再重复介绍这些寄存器。下面将重点介绍在 PCI 桥中与 PCI Agent 的配置空间不相同的寄存器。

与 PCI Agent 设备不同, 在 PCI 桥中只含有两组 BAR 寄存器, 即 Base Address Register 0~1 寄存器。这两组寄存器与 PCI Agent 设备配置空间的对应寄存器的含义一致。但是在 PCI 桥中, 这两组寄存器是可选的。如果在 PCI 桥中不存在私有寄存器, 那么可以不使用这两组寄存器设置 BAR 空间。

在大多数 PCI 桥中都不存在私有寄存器, 操作系统也不需要为 PCI 桥提供专门的驱动程序, 这也是这类桥被称为透明桥的原因。如果在 PCI 桥中不存在私有空间时, PCI 桥将这两个 BAR 寄存器初始化为 0。在 PCI 桥的配置空间中使用两个 BAR 寄存器的原因是这两个 32 位的寄存器可以组成一个 64 位地址空间。

在 PCI 桥的配置空间中, 有许多寄存器是 PCI 桥所特有的。PCI 桥除了作为 PCI 设备之

⊖ 此时 GNT#信号为无效。为提高仲裁效率, PCI 设备在进行数据传送时, GNT#信号可能已经无效。

外，还需要管理其下连接的 PCI 总线子树使用的各类资源，即 Secondary Bus 所连接 PCI 总线子树使用的资源。这些资源包括存储器、I/O 地址空间和总线号。

31	24	23	16	15	8	7	0	
Device ID				Vendor ID				0x00
Status				Command				0x04
Class Code						Revision ID		0x08
BIST		Header Type		Primary Latency Timer		Cache Line Size		0x0C
Base Address Register0								0x10
Base Address Register1								0x14
Secondary Latency Timer		Subordinate Bus Number		Secondary Bus Number		Primary Bus Number		0x18
Secondary Status				I/O Limit		I/O Base		0x1C
Memory Limit				Memory Base				0x20
Prefetchable Memory Limit				Prefetchable Memory Base				0x24
Prefetchable Base Upper 32 Bits								0x28
Prefetchable Limit Upper 32 Bits								0x2C
I/O Limit Upper 16 bits				I/O Base Upper 16 bits				0x30
Reserved						Capabilities Pointer		0x34
Expansion ROM Base Address								0x38V
Bridge Control				Interrupt Pin		Interrupt Line		0x3C

图 2-10 PCI 桥的配置空间

在 PCI 桥中，与 Secondary Bus 相关的寄存器包括两大类。一类寄存器管理 Secondary Bus 之下 PCI 子树的总线号，如 Secondary 和 Subordinate Bus Number 寄存器；另一类寄存器管理下游 PCI 总线的 I/O 和存储器地址空间，如 I/O 和 Memory Limit、I/O 和 Memory Base 寄存器。在 PCI 桥中还使用 Primary Bus 寄存器保存上游的 PCI 总线号。

其中存储器地址空间还分为可预读空间和不可预读空间，Prefetchable Memory Limit 和 Prefetchable Memory Base 寄存器管理可预读空间，而 Memory Limit、Memory Base 管理不可预读空间。在 PCI 体系结构中，除了 ROM 地址空间之外，PCI 设备使用的地址空间大多都是不可预读的。

(1) Subordinate Bus Number、Secondary Bus Number 和 Primary Bus Number 寄存器

PCI 桥可以管理其下的 PCI 总线子树。其中 Subordinate Bus Number 寄存器存放当前 PCI 子树中，编号最大的 PCI 总线号。而 Secondary Bus Number 寄存器存放当前 PCI 桥 Secondary Bus 使用的总线号，这个 PCI 总线号也是该 PCI 桥管理的 PCI 子树中编号最小的 PCI 总线号。因此一个 PCI 桥能够管理的 PCI 总线号在 Secondary Bus Number~Subordinate Bus Number 之间。这两个寄存器的值由系统软件遍历 PCI 总线树时设置。

Primary Bus Number 寄存器存放该 PCI 桥上游的 PCI 总线号，该寄存器可读写。Primary Bus Number、Subordinate Bus Number 和 Secondary Bus Number 寄存器在初始化时必须为 0，系统软件将根据这几个寄存器是否为 0，判断 PCI 桥是否被配置过。

不同的操作系统使用不同的 Bootloader 引导，有的 Bootloader 可能会对 PCI 总线树进行遍历，此时操作系统不必重新遍历 PCI 总线树。在 x86 处理器系统中，BIOS 会遍历处理器系统中的所有 PCI 总线树，操作系统可以直接使用 BIOS 的结果，也可以重新遍历 PCI 总线树。而 PowerPC 处理器系统中的 Bootloader，如 U-Boot 并没有完全遍历 PCI 总线树，此时操作系统必须重新遍历 PCI 总线树。本书将在第 14 章以 Linux 系统为例说明 PCI 总线树的遍

历过程。

(2) Secondary Status 寄存器

该寄存器的含义与 PCI Agent 配置空间的 Status 寄存器的含义相近，PCI 桥的 Secondary Status 寄存器记录 Secondary Bus 的状态，而不是 PCI 桥作为 PCI 设备时使用的状态。在 PCI 桥配置空间中还存在一个 Status 寄存器，该寄存器保存 PCI 桥作为 PCI 设备时的状态。

(3) Secondary Latency Timer 寄存器

该寄存器的含义与 PCI Agent 配置空间的 Latency Timer 寄存器的含义相近，PCI 桥的 Secondary Latency Timer 寄存器管理 Secondary Bus 的超时机制，即 PCI 桥发向下游的总线事务；在 PCI 桥配置空间中还存在一个 Latency Timer 寄存器，该寄存器管理 PCI 桥发向上游的总线事务。

(4) I/O Limit 和 I/O Base 寄存器

在 PCI 桥管理的 PCI 子树中包含许多 PCI 设备，而这些 PCI 设备可能会使用 I/O 地址空间。PCI 桥使用这两个寄存器，存放 PCI 子树中所有设备使用的 I/O 地址空间集合的基地址和大小。

(5) Memory Limit 和 Memory Base 寄存器

在 PCI 桥管理的 PCI 子树中有许多 PCI 设备，这些 PCI 设备可能会使用存储器地址空间。这两个寄存器存放所有这些 PCI 设备使用的存储器地址空间集合的基地址和大小，PCI 桥规定这个空间的大小至少为 1MB。

(6) Prefetchable Memory Limit 和 Prefetchable Memory Base 寄存器

在 PCI 桥管理的 PCI 子树中有许多 PCI 设备，如果这些 PCI 设备支持预读，则需要从 PCI 桥的可预读空间中获取地址空间。PCI 桥的这两个寄存器存放这些 PCI 设备使用的可预取存储器空间的基地址和大小。

如果 PCI 桥不支持预读，则其下支持预读的 PCI 设备需要从 Memory Base 寄存器为基地址的存储器空间中获取地址空间。如果 PCI 桥支持预读，其下的 PCI 设备需要根据情况，决定使用可预读空间还是不可预读空间。PCI 总线建议 PCI 设备支持预读，但是支持预读的 PCI 设备并不多见。

(7) I/O Base Upper 16 bits 和 I/O Limit Upper 16 bits 寄存器

如果 PCI 桥仅支持 16 位的 I/O 端口，这组寄存器只读，且其值为 0。如果 PCI 桥支持 32 位 I/O 端口，这组寄存器可以提供 I/O 端口的高 16 位地址。

(8) Bridge Control 寄存器

该寄存器用来管理 PCI 桥的 Secondary Bus，其主要位的描述如下。

- Secondary Bus Reset 位，第 6 位，可读写。当该位为 1 时，将使用下游总线提供的 RST#信号复位与 PCI 桥的下游总线连接的 PCI 设备。通常情况下与 PCI 桥下游总线连接的 PCI 设备，其复位信号需要与 PCI 桥提供的 RST#信号连接，而不能与 HOST 主桥提供的 RST#信号连接。
- Primary Discard Timer 位，第 8 位，可读写。PCI 桥支持 Delayed 传送方式，当 PCI 桥的 Primary 总线上的主设备使用 Delayed 方式进行数据传递时，PCI 桥使用 Retry 周期结束 Primary 总线的 Non-Posted 数据请求，并将这个 Non-Posted 数据请求转换为 Delayed 数据请求，之后主设备需要择时重试相同的 Non-Posted 数据请求。当该位为 1

时，表示在 Primary Bus 上的主设备需要在 2^{10} 个时钟周期之内重试这个数据请求，为 0 时，表示主设备需要在 2^{15} 个时钟周期之内重试这个数据请求，否则 PCI 桥将丢弃 Delayed 数据请求。

- Secondary Discard Timer 位，第 9 位，可读写。当该位为 1 时，表示在 Secondary Bus 上的主设备需要在 2^{10} 个时钟周期之内重试这个数据请求，为 0 时，表示主设备需要在 2^{15} 个时钟周期之内重试这个数据请求，如果主设备在规定的时间内没有进行重试时，PCI 桥将丢弃 Delayed 数据请求。

2.4 PCI 总线的配置

PCI 总线定义了两类配置请求，一类是 Type 00h 配置请求，另一类是 Type 01h 配置请求。PCI 总线使用这些配置请求访问 PCI 总线树上的设备配置空间，包括 PCI 桥和 PCI Agent 设备的配置空间。

其中 HOST 主桥或者 PCI 桥使用 Type 00h 配置请求，访问与 HOST 主桥或者 PCI 桥直接相连的 PCI Agent 设备或者 PCI 桥^①；而 HOST 主桥或者 PCI 桥使用 Type 01h 配置请求，需要至少穿越一个 PCI 桥，访问没有与其直接相连的 PCI Agent 设备或者 PCI 桥。如图 2-8 所示，HOST 主桥可以使用 Type 00h 配置请求访问 PCI 设备 01，而使用 Type 01h 配置请求通过 PCI 桥 1、2 或者 3 转换为 Type 00h 配置请求之后，访问 PCI 总线树上的 PCI 设备 11、21、22、31 和 32^②。

当 x86 处理器对 CONFIG_DATA 寄存器进行读写操作时，HOST 主桥将决定向 PCI 总线发送 Type 00h 配置请求还是 Type 01h 配置请求。在 PCI 总线事务的地址周期中，这两种配置请求总线事务的不同反映在 PCI 总线的 AD [31:0] 信号线上。

值得注意的是，PCIe 总线还可以使用 ECAM (Enhanced Configuration Access Mechanism) 机制访问 PCIe 设备的扩展配置空间，使用这种方式可以访问 PCIe 设备 256B~4 KB 之间的扩展配置空间。但是本节仅介绍如何使用 CONFIG_ADDRESS 和 CONFIG_FATA 寄存器产生 Type 00h 和 Type 01h 配置请求。有关 ECAM 机制的详细说明见第 5.3.2 节。

处理器首先将目标 PCI 设备的 ID 号保存在 CONFIG_ADDRESS 寄存器中，之后 HOST 主桥根据该寄存器的 Bus Number 字段，决定是产生 Type 00h 配置请求，还是 Type 01h 配置请求。当 Bus Number 字段为 0 时，将产生 Type 00h 配置请求，因为与 HOST 主桥直接相连的总线号为 0；大于 0 时，将产生 Type 01h 配置请求。

2.4.1 Type 01h 和 Type 00h 配置请求

本节首先介绍 Type 01h 配置请求，并从 PCI 总线使用的信号线的角度上，讲述 HOST 主桥如何生成 Type 01 配置请求。在 PCI 总线中，只有 PCI 桥能够接收 Type 01h 配置请求。Type 01h 配置请求不能直接发向最终的 PCI Agent 设备，而只能由 PCI 桥将其转换为 Type 01h 继续发向其他 PCI 桥，或者转换为 Type 00h 配置请求发向 PCI Agent 设备。PCI 桥还可

① 此时 PCI 桥作为一个 PCI 设备，接收访问其配置空间的读写请求。

② 最终 Type 01h 配置请求将会被转换为 Type 00h 配置请求，然后访问 PCI Agent 设备。

以将 Type 01h 配置请求转换为 Special Cycle 总线事务（HOST 主桥也可以实现该功能），本节对这种情况不做介绍。

在地址周期中，HOST 主桥使用配置读写总线事务，将 CONFIG_ADDRESS 寄存器的内容复制到 PCI 总线的 AD [31:0] 信号线中。CONFIG_ADDRESS 寄存器与 Type 01h 配置请求的对应关系如图 2-11 所示。

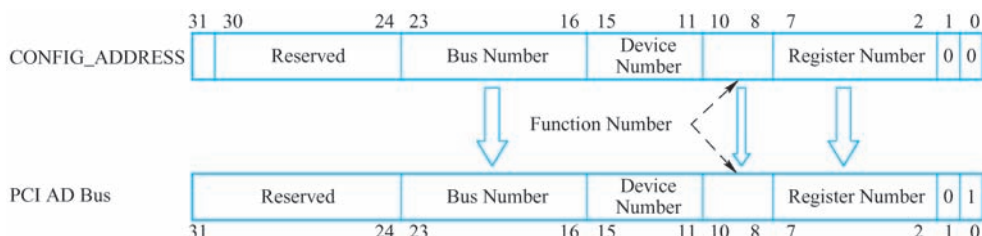


图 2-11 CONFIG_ADDRESS 寄存器与 Type 01h 配置请求的对应关系

从图 2-11 中可以发现，CONFIG_ADDRESS 寄存器的内容基本上是原封不动地复制到 PCI 总线的 AD [31:0] 信号线上的^①。其中 CONFIG_ADDRESS 的使能位不被复制，而 AD 总线的第 0 位必须为 1，表示当前配置请求是 Type 01h。

当 PCI 总线接收到 Type 01 配置请求时，将寻找合适的 PCI 桥^②接收这个配置信息。如果这个配置请求是直接发向 PCI 桥下的 PCI 设备时，PCI 桥将接收这个 Type 01 配置请求，并将其转换为 Type 00h 配置请求；否则 PCI 桥将当前 Type 01h 配置请求原封不动地传递给下一级 PCI 总线。

如果 HOST 主桥或者 PCI 桥发起的是 Type 00h 配置请求，CONFIG_ADDRESS 寄存器与 AD [31:0] 的转换如图 2-12 所示。

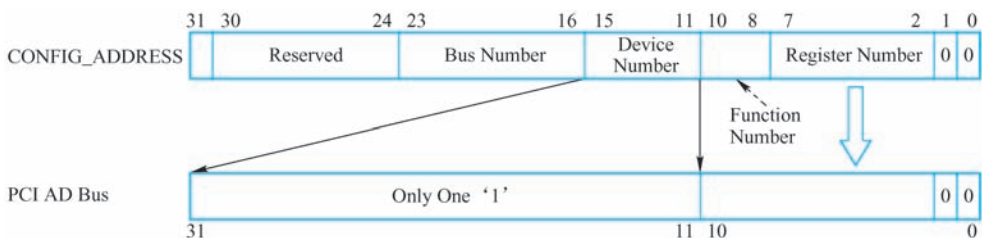


图 2-12 CONFIG_ADDRESS 寄存器与 Type 00h 配置请求的对应关系

此时处理器对 CONFIG_DATA 寄存器进行读写时，处理器将 CONFIG_ADDRESS 寄存器中的 Function Number 和 Register Number 字段复制到 PCI 的 AD 总线的第 10~2 位；将 AD 总线的第 1~0 位赋值为 0b00。PCI 总线在配置请求总线事务的地址周期根据 AD [1:0] 判断当前配置请求是 Type 00h 还是 Type 01h，如果 AD [1:0] 等于 0b00 表示是 Type 00h 配置请求，如果 AD [1:0] 等于 0b01 表示是 Type 01h 配置请求。

而 AD [31:11] 与 CONFIG_ADDRESS 的 Device Number 字段有关，在 Type 00h 配置请

① Type 01h 配置头信息存在于 PCI 总线事务的地址周期中。

② PCI 桥根据 Subordinate Bus Number 和 Secondary Bus Number 寄存器，决定是否接收当前配置请求。

求的地址周期中，AD [31:11] 位有且只有一位为 1，其中 AD [31:11] 的每一位选通一个 PCI 设备的配置空间。如第 1.2.2 节所述，PCI 设备配置空间的片选信号是 IDSEL，因此 AD [31:11] 将与 PCI 设备的 IDSEL 信号对应相连。

当以下两种请求之一满足时，HOST 主桥或者 PCI 桥将生成 Type 00h 配置头，并将其发送到指定的 PCI 总线上。

1) CONFIG_ADDRESS 寄存器的 Bus Number 字段为 0 时，处理器访问 CONFIG_DATA 寄存器时，HOST 主桥将直接向 PCI 总线 0 发出 Type 00h 配置请求。因为与 HOST 主桥直接相连的 PCI 总线号为 0，此时表示 HOST 主桥需要访问与其直接相连的 PCI 设备。

2) 当 PCI 桥收到 Type 01h 配置头时，将检查 Type 01 配置头的 Bus Number 字段，如果这个 Bus Number 与 PCI 桥的 Secondary Bus Number 相同，则将这个 Type 01 配置头转换为 Type 00h 配置头，并发送到该 PCI 桥的 Secondary 总线上。

2.4.2 PCI 总线配置请求的转换原则

当 CONFIG_ADDRESS 寄存器的 Enable 位为 1，系统软件访问 CONFIG_DATA 寄存器时，HOST 主桥可以产生两类 PCI 总线配置读写总线事务，分别为 Type 00h 和 Type 01h 配置读写总线事务。在配置读写总线事务的地址周期和数据周期中，CONFIG_ADDRESS 和 CONFIG_DATA 寄存器中的数据将被放置到 PCI 总线的 AD 总线上。其中 Type 00h 和 Type 01h 配置读写总线事务映射到 AD 总线的数据并不相同。

其中 Type 00h 配置请求可以直接读取 PCI Agent 设备的配置空间，而 Type 01h 配置请求在通过 PCI 桥时，最终将被转换为 Type 00h 配置请求，并读取 PCI Agent 设备的配置寄存器。本节重点讲述 PCI 桥如何将 Type 01h 配置请求转换为 Type 00h 配置请求。

首先 Type 00h 配置请求不会被转换成 Type 01h 配置请求，因为 Type 00h 配置请求是发向最终 PCI Agent 设备，这些 PCI Agent 设备不会转发这些配置请求。

当 CONFIG_ADDRESS 寄存器的 Bus Number 字段为 0 时，处理器对 CONFIG_DATA 寄存器操作时，HOST 主桥将直接产生 Type 00h 配置请求，挂接在 PCI 总线 0 上的某个设备将通过 ID 译码接收这个 Type 00h 配置请求，并对配置寄存器进行读写操作。如果 PCI 总线上没有设备接收这个 Type 00h 配置请求，将引发 Master Abort，详情见 PCI 总线规范，本节对此不做进一步说明。

如果 CONFIG_ADDRESS 寄存器的 Bus Number 字段为 n ($n \neq 0$)，即访问的 PCI 设备不是直接挂接在 PCI 总线 0 上的，此时 HOST 主桥对 CONFIG_DATA 寄存器操作时，将产生 Type 01h 配置请求，PCI 总线 0 将遍历所有在这条总线上的 PCI 桥，确定由哪个 PCI 桥接收这个 Type 01h 配置请求。

如果 n 大于或等于某个 PCI 桥的 Secondary Bus Number 寄存器，而且小于或等于 Subordinate Bus number 寄存器，那么这个 PCI 桥将接收在当前 PCI 总线上的 Type 01 配置请求，并采用以下规则进行递归处理。

1) 开始。

2) 遍历当前 PCI 总线的所有 PCI 桥。

3) 如果 n 等于某个 PCI 桥的 Secondary Bus Number 寄存器，说明这个 Type 01 配置请求的目标设备直接连接在该 PCI 桥的 Secondary bus 上。此时 PCI 桥将 Type 01 配置请求转

换为 Type 00h 配置请求，并将这个配置请求发送到 PCI 桥的 Secondary Bus 上，Secondary Bus 上的某个设备将响应这个 Type 00h 配置请求，并与 HOST 主桥进行配置信息的交换，转 5)。

4) 如果 n 大于 PCI 桥的 Secondary Bus Number 寄存器，而且小于或等于 PCI 桥的 Subordinate Bus number 寄存器，说明这个 Type 01 配置请求的目标设备不与该 PCI 桥的 Secondary Bus 直接相连，但是由这个 PCI 桥下游总线上的某个 PCI 桥管理。此时 PCI 桥将首先认领这个 Type 01 配置请求，并将其转发到 Secondary Bus，转 2)。

5) 结束。

下面将举例说明 PCI 总线配置请求的转换原则，并以图 2-8 为例说明处理器如何访问 PCI 设备 01 和 PCI 设备 31 的配置空间。PCI 设备 01 直接与 HOST 主桥相连，因此 HOST 主桥可以使用 Type 00h 配置请求访问该设备。

而 HOST 主桥需要经过多级 PCI 桥才能访问 PCI 设备 31，因此 HOST 主桥需要首先使用 Type 01h 配置请求，之后通过 PCI 桥 1、2 和 3 将 Type 01h 配置请求转换为 Type 00h 配置请求，最终访问 PCI 设备 31。

1. PCI 设备 01

这种情况较易处理，当 HOST 处理器访问 PCI 设备 01 的配置空间时，发现 PCI 设备 01 与 HOST 主桥直接相连，所以将直接使用 Type 00h 配置请求访问该设备的配置空间，具体步骤如下。

首先 HOST 处理器将 CONFIG_ADDRESS 寄存器的 Enabled 位置 1，Bus Number 号置为 0，并对该寄存器的 Device、Function 和 Register Number 字段赋值。当处理器对 CONFIG_DATA 寄存器访问时，HOST 主桥将存放在 CONFIG_ADDRESS 寄存器中的数值，转换为 Type 00h 配置请求，并发送到 PCI 总线 0 上，PCI 设备 01 将接收这个 Type 00h 配置请求，并与处理器进行配置信息交换。

2. PCI 设备 31

HOST 处理器对 PCI 设备 31 进行配置读写时，需要通过 HOST 主桥、PCI 桥 1、2 和 3，最终到达 PCI 设备 31。

当处理器访问 PCI 设备 31 时，首先将 CONFIG_ADDRESS 寄存器的 Enabled 位置 1，Bus Number 字段置为 3，并对 Device、Function 和 Register Number 字段赋值。之后当处理器对 CONFIG_DATA 寄存器进行读写访问时，HOST 主桥、PCI 桥 1、2 和 3 将按照以下步骤进行处理，最后 PCI 设备 31 将接收这个配置请求。

1) HOST 主桥发现 Bus Number 字段的值为 3，该总线号并不是与 HOST 主桥直接相连的 PCI 总线的 Bus Number，所以 HOST 主桥将处理器对 CONFIG_DATA 寄存器的读写访问直接转换为 Type 01h 配置请求，并将这个配置请求发送到 PCI 总线 0 上。PCI 总线规定 Type 01h 配置请求只能由 PCI 桥负责处理。

2) 在 PCI 总线 0 上，PCI 桥 1 的 Secondary Bus Number 为 1 而 Subordinate Bus Number 为 3。而 $1 < \text{Bus Number} \leq 3$ ，所以 PCI 桥 1 将接收来自 PCI 总线 0 的 Type 01h 配置请求，并将这个配置请求直接下推到 PCI 总线 1。

3) 在 PCI 总线 1 上，PCI 桥 2 的 Secondary Bus Number 为 2 而 Subordinate Bus Number 为 3。而 $1 < \text{Bus Number} \leq 3$ ，所以 PCI 桥 2 将接收来自 PCI 总线 0 的 Type 01h 配置请求，并将

这个配置请求直接下推到 PCI 总线 2。

4) 在 PCI 总线 2 上, PCI 桥 3 的 Secondary Bus Number 为 3, 因此 PCI 桥 3 将“来自 PCI 总线 2 的 Type 01h 配置请求”转换为 Type 00h 配置请求, 并将其下推到 PCI 总线 3。PCI 总线规定, 如果 PCI 桥的 Secondary Bus Number 与 Type 01h 配置请求中包含的 Bus Number 相同时, 该 PCI 桥将接收的 Type 01h 配置请求转换为 Type 00h 配置请求, 然后再发向其 Secondary Bus。

5) 在 PCI 总线 3 上, 有两个设备: PCI 设备 31 和 PCI 设备 32。在这两个设备中, 必然有一个设备将要响应这个 Type 00h 配置请求, 从而完成整个配置请求周期。在第 2.4.1 节中, 讨论了究竟是 PCI 设备 31 还是 PCI 设备 32 接收这个配置请求, 这个问题涉及 PCI 总线如何分配 PCI 设备使用的设备号。

2.4.3 PCI 总线树 Bus 号的初始化

在一个处理器系统中, 每一个 HOST 主桥都推出一棵 PCI 总线树。在一棵 PCI 总线树中有多少个 PCI 桥 (包括 HOST 主桥), 就含有多少条 PCI 总线。系统软件在遍历当前 PCI 总线树时, 需要首先对这些 PCI 总线进行编号, 即初始化 PCI 桥的 Primary、Secondary 和 Subordinate Bus Number 寄存器。

在一个处理器系统中, 一般将与 HOST 主桥直接相连的 PCI 总线命名为 PCI 总线 0。然后系统软件使用 DFS (Depth First Search) 算法, 依次对其他 PCI 总线进行编号。值得注意的是, 与 HOST 主桥直接相连的 PCI 总线, 其编号都为 0, 因此当处理器系统中存在多个 HOST 主桥时, 将有多条编号为 0 的 PCI 总线, 但是这些编号为 0 的 PCI 总线分属不同的 PCI 总线域, 其含义并不相同。

在一个处理器系统中, 假设 PCI 总线树的结构如图 2-13 所示。当然在一个实际的处理器系统中, 很少会出现这样复杂的 PCI 总线树结构, 本节采用这个结构的目的是便于说明 PCI 总线号的分配过程。

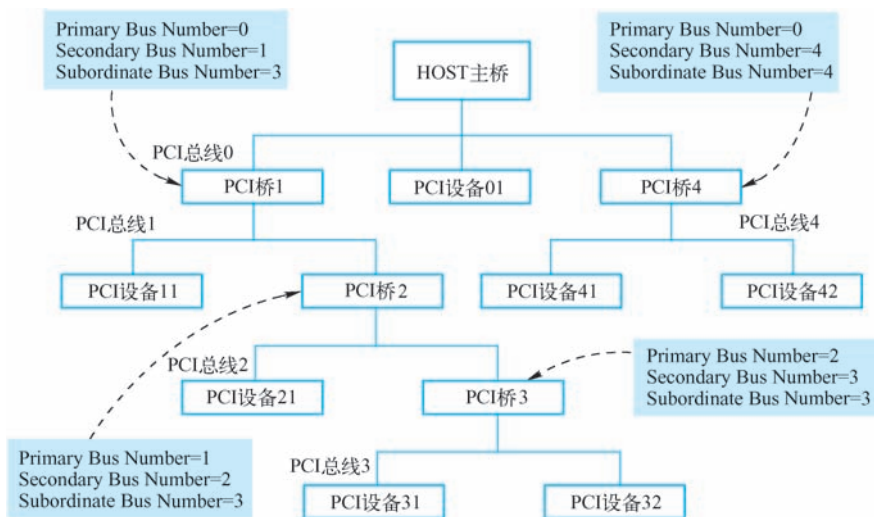


图 2-13 PCI 总线树结构

在 PCI 总线中，系统软件使用深度优先 DFS 算法对 PCI 总线树进行遍历，DFS 算法和广度优先 BFS（Breadth First Search）算法是遍历树型结构的常用算法。与 BFS 算法相比，DFS 算法的空间复杂度较低，因此绝大多数系统在遍历 PCI 总线树时，都使用 DFS 算法而不是 BFS 算法。

DFS 是搜索算法的一种，其实现机制是沿着一棵树的深度遍历各个节点，并尽可能深地搜索树的分支，DFS 的算法为线性时间复杂度，适合对拓扑结构未知的树进行遍历。在一个处理器系统的初始化阶段，PCI 总线树的拓扑结构是未知的，适合使用 DFS 算法进行遍历。下面以图 2-13 为例，说明系统软件如何使用 DFS 算法，分配 PCI 总线号，并初始化 PCI 桥中的 Primary Bus Number、Secondary Bus Number 和 Subordinate Bus Number 寄存器。所谓 DFS 算法是指按照深度优先的原则遍历 PCI 树，其步骤如下。

1) HOST 主桥扫描 PCI 总线 0 上的设备。系统软件首先忽略这条总线上的所有 PCI Agent 设备，因为在这些设备之下不会挂接新的 PCI 总线。例如 PCI 设备 01 下不可能挂接新的 PCI 总线。

2) HOST 主桥首先发现 PCI 桥 1，并将 PCI 桥 1 的 Secondary Bus 命名为 PCI 总线 1。系统软件将初始化 PCI 桥 1 的配置空间，将 PCI 桥 1 的 Primary Bus Number 寄存器赋值为 0，而将 Secondary Bus Number 寄存器赋值为 1，即 PCI 桥 1 的上游 PCI 总线号为 0，而下游 PCI 总线号为 1。

3) 扫描 PCI 总线 1，发现 PCI 桥 2，并将 PCI 桥 2 的 Secondary Bus 命名为 PCI 总线 2。系统软件将初始化 PCI 桥 2 的配置空间，将 PCI 桥 2 的 Primary Bus Number 寄存器赋值为 1，而将 Secondary Bus Number 寄存器赋值为 2。

4) 扫描 PCI 总线 2，发现 PCI 桥 3，并将 PCI 桥 3 的 Secondary Bus 命名为 PCI 总线 3。系统软件将初始化 PCI 桥 3 的配置空间，将 PCI 桥 3 的 Primary Bus Number 寄存器赋值为 2，而将 Secondary Bus Number 寄存器赋值为 3。

5) 扫描 PCI 总线 3，没有发现任何 PCI 桥，这表示 PCI 总线 3 下不可能有新的总线，此时系统软件将 PCI 桥 3 的 Subordinate Bus Number 寄存器赋值为 3。系统软件在完成 PCI 总线 3 的扫描后，将回退到 PCI 总线 3 的上一级总线，即 PCI 总线 2，继续进行扫描。

6) 在重新扫描 PCI 总线 2 时，系统软件发现 PCI 总线 2 上除了 PCI 桥 3 之外没有发现新的 PCI 桥，而 PCI 桥 3 之下的所有设备已经完成了扫描过程，此时系统软件将 PCI 桥 2 的 Subordinate Bus Number 寄存器赋值为 3。继续回退到 PCI 总线 1。

7) PCI 总线 1 上除了 PCI 桥 2 外，没有其他桥片，于是继续回退到 PCI 总线 0，并将 PCI 桥 1 的 Subordinate Bus Number 寄存器赋值为 3。

8) 在 PCI 总线 0 上，系统软件扫描到 PCI 桥 4，则首先将 PCI 桥 4 的 Primary Bus Number 寄存器赋值为 0，而将 Secondary Bus Number 寄存器赋值为 4，即 PCI 桥 1 的上游 PCI 总线号为 0，而下游 PCI 总线号为 4。

9) 系统软件发现 PCI 总线 4 上没有任何 PCI 桥，将结束对 PCI 总线 4 的扫描，并将 PCI 桥 4 的 Subordinate Bus Number 寄存器赋值为 4，之后回退到 PCI 总线 4 的上游总线，即 PCI 总线 0 继续进行扫描。

10) 系统软件发现在 PCI 总线 0 上的两个桥片 PCI 总线 0 和 PCI 总线 4 都已完成扫描后，将结束对 PCI 总线的 DFS 遍历全过程。

从以上算法可以看出，PCI 桥的 Primary Bus 和 Secondary Bus 号的分配在遍历 PCI 总线树的过程中从上向下分配，而 Subordinate Bus 号是从下向上分配的，因为只有确定了一个 PCI 桥之下究竟有多少条 PCI 总线后，才能初始化该 PCI 桥的 Subordinate Bus 号。

2.4.4 PCI 总线 Device 号的分配

一条 PCI 总线会挂接各种各样的 PCI 设备，而每一个 PCI 设备在 PCI 总线下具有唯一的设备号。系统软件通过总线号和设备号定位一个 PCI 设备之后，才能访问这个 PCI 设备的配置寄存器。值得注意的是，系统软件使用“地址寻址方式”访问 PCI 设备的存储器和 I/O 地址空间，这与访问配置空间使用的“ID 寻址方式”不同。

PCI 设备的 IDSEL 信号与 PCI 总线的 AD [31:0] 信号的连接关系决定了该设备在这条 PCI 总线的设备号。如上文所述，每一个 PCI 设备都使用独立的 IDSEL 信号，该信号将与 PCI 总线的 AD [31:0] 信号连接，IDSEL 信号的含义见第 1.2.2 节。

在此我们简要回顾 PCI 的配置读写事务使用的时序。如图 1-3 所示，PCI 总线事务由一个地址周期加若干个数据周期组成。在进行配置读写请求总线事务时，C/BE# 信号线的值在地址周期中为 0x1010 或者为 0x1011，表示当前总线事务为配置读或者配置写请求。此时出现在 AD [31:0] 总线上的值并不是目标设备的 PCI 总线地址，而是目标设备的 ID 号，这与 PCI 总线进行 I/O 或者存储器请求时不同，因为 PCI 总线使用 ID 号而不是 PCI 总线地址对配置空间进行访问。

如图 2-12 所示，在配置读写总线事务的地址周期中，AD [10:0] 信号已经被 Function Number 和 Register Number 使用，因此 PCI 设备的 IDSEL 只能与 AD [31:11] 信号连接。

认真的读者一定可以发现在 CONFIG_ADDRESS 寄存器中 Device Number 字段一共有 5 位可以表示 32 个设备，而 AD [31:11] 只有 21 位，显然在这两者之间无法建立一一对应的映射关系。因此在一条 PCI 总线上如果有 21 个以上的 PCI 设备，那么总是有几个设备无法与 AD [31:11] 信号线连接，从而 PCI 总线无法访问这些设备。因为 PCI 总线在配置请求的地址周期中，只能使用第 31~11 这些 AD 信号，所以在一条总线上最多也只能挂接 21 个 PCI 设备。这 21 个设备可能是从 0 到 20，也可能是从 11 到 31 排列。从而系统软件在遍历 PCI 总线时，还是需要从 0 到 31 遍历整条 PCI 总线。

在实际的应用中，一条 PCI 总线能够挂接 21 个设备已经足够了，实际上由于 PCI 总线的负载能力有限，即便在总线频率为 33 MHz 的情况下，在一条 PCI 总线中最多也只能挂接 10 个负载，一条 PCI 总线所能挂接的负载详见表 1-1。AD 信号线与 PCI 设备 IDSEL 线的连接关系如图 2-14 所示。

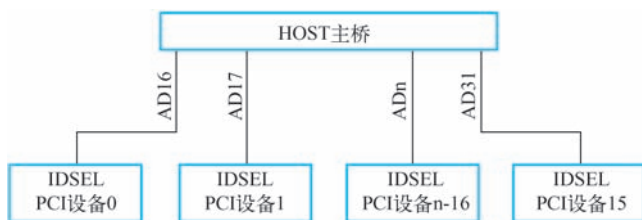


图 2-14 PCI 总线设备号的分配

PCI 总线推荐了一种 Device Number 字段与 AD [31:16] 之间的映射关系。其中 PCI 设备 0 与 Device Number 字段的 0b00000 对应；PCI 设备 1 与 Device Number 字段的 0b00001 对应，并以此类推，PCI 设备 15 与 Device Number 字段的 0b01111 对应。

在这种映射关系之下，一条 PCI 总线中，与信号线 AD16 相连的 PCI 设备的设备号为 0；与信号线 AD17 相连的 PCI 设备的设备号为 1；以此类推，与信号线 AD31 相连的 PCI 设备的设备号为 15。在 Type 00h 配置请求中，设备号并没有像 Function Number 和 Register Number 那样以编码的形式出现在 AD 总线上，而是与 AD 信号一一对应，如图 2-12 所示。

这里有一个原则需要读者注意，就是对 PCI 设备的配置寄存器进行访问时，一定要有确定的 Bus Number、Device Number、Function Number 和 Register Number，这“四元组”缺一不可。在 Type 00h 配置请求中，Device Number 由 AD [31:11] 信号线与 PCI 设备 IDSEL 信号的连接关系确定；Function Number 保存在 AD [10:8] 字段中；而 Register Number 保存在 AD [7:0] 字段中；在 Type 01h 配置请求中，也有完整的四元组信息。

在一个处理器系统的设计中，如果在一条 PCI 总线上使用的 PCI 插槽少于 4 个时，笔者建议优先使用 AD [17:20] 信号与 PCI 设备的 IDSEL 信号连接。因为 PCI-X 总线规范建议使用 AD17 连接 PCI 设备 1、AD18 连接 PCI 设备 2、AD19 连接 PCI 设备 3、AD20 连接 PCI 设备 4，采用这种方法便于实现 PCI 总线与 PCI-X 总线的兼容。

2.5 非透明 PCI 桥

PCI 桥规范定义了透明桥的实现规则，在第 2.3.1 节中详细介绍了这种桥片。通过透明桥，处理器系统可以以 HOST 主桥为根节点，建立一颗 PCI 总线树，在这个树上的 PCI 设备共享同一个 PCI 总线域上的地址空间。

但是在某些场合下 PCI 透明桥并不适用。在图 2-15 所示的处理器系统中存在两个处理器，此时使用 PCI 桥 1 连接处理器 2 并不利于整个处理器系统的配置与管理。假定 PCI 总线使用 32 位地址空间，而处理器 1 和处理器 2 所使用的存储器大小都为 2 GB，同时假定处理器 1 和处理器 2 使用的存储器都可以被 PCI 设备访问。

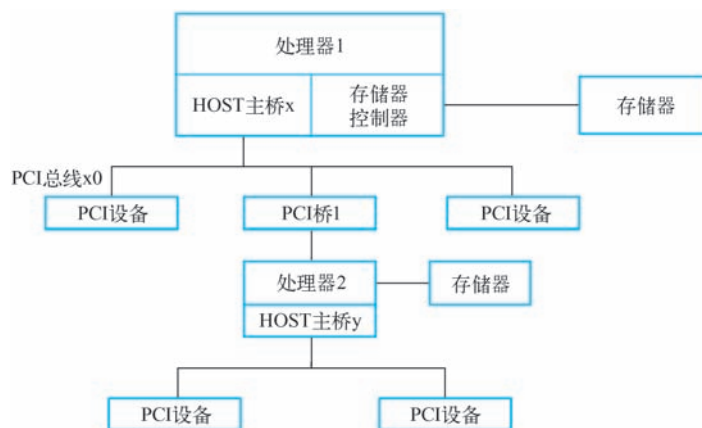


图 2-15 不适合 PCI 透明桥的处理器系统互连方式

此时处理器 1 和 2 使用的存储器空间必须映射到 PCI 总线的地址空间中，而 32 位的 PCI 总线只能提供 4GB 地址空间，此时 PCI 总线 x0 的地址空间将全部被处理器 1 和 2 的存储器空间占用，而没有额外的空间分配给 PCI 设备。

此外有些处理器不能作为 PCI Agent 设备，因此不能直接连接到 PCI 桥上，比如 x86 处理器就无法作为 PCI Agent 设备，因此使用 PCI 透明桥无法将两个 x86 处理器直接相连。如果处理器 2 有两个以上的 PCI 接口，其中一个可以与 PCI 桥 1 相连（此时处理器 2 将作为 PCI Agent 设备），而另一个作为 HOST 主桥 y 连接 PCI 设备。此时 HOST 主桥 y 挂接的 PCI 设备将无法被处理器 1 直接访问。

使用透明桥也不便于解决处理器 1 与处理器 2 间的地址冲突。对于图 2-15 所示的处理器系统，如果处理器 1 和 2 都将各自的存储器映射到 PCI 总线地址空间中，有可能出现地址冲突。虽然 PowerPC 处理器可以使用 Inbound 寄存器，将存储器地址空间映射到不同的 PCI 总线地址空间中，但并非所有的处理器都具有这种映射机制。许多处理器的存储器地址与 PCI 总线地址使用了“简单相等”这种映射方法，如果 PCI 总线连接了两个这样的处理器，将不可避免地出现 PCI 总线地址的映射冲突。

采用非透明桥将有效解决以上这些问题，非透明桥并不是 PCI 总线定义的标准桥片，但是这类桥片在连接两个处理器系统中得到了广泛的应用。一个使用非透明桥连接两个处理器系统的实例如图 2-16 所示。

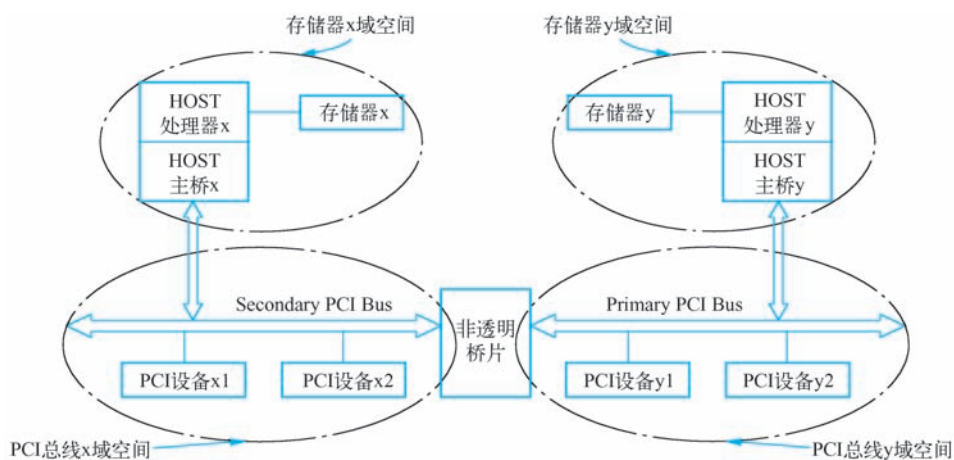


图 2-16 使用 PCI 非透明桥连接两个处理器系统

使用非透明 PCI 桥可以方便地连接两个处理器系统。从图 2-16 中我们可以发现非透明桥可以将 PCI 总线 x 域与 PCI 总线 y 域进行隔离。值得注意的是，非透明 PCI 桥的作用是对不同 PCI 总线域地址空间进行隔离，而不是隔离存储器域地址空间。而 HOST 主桥的作用才是将存储器域与 PCI 总线域进行隔离。

非透明 PCI 桥可以连接两条独立的 PCI 总线，一条被称为 Secondary PCI 总线，另一条被称为 Primary PCI 总线，但是这两条总线没有从属关系，两边是对等的^①。从处理器 x 的角

① 有些非透明桥，如 DEC21554 的两边并不是完全对等的，尤其是在处理 64 位地址空间时，本文对此不做详细说明。

度来看，与非透明 PCI 桥右边连接的总线叫 Secondary PCI 总线；而从处理器 y 的角度来看，非透明 PCI 桥左边连接的总线叫 Secondary PCI 总线。

HOST 处理器 x 和 PCI 设备可以通过非透明 PCI 桥直接访问 PCI 总线 y 域的地址空间，并通过 HOST 主桥 y 访问存储器 y；HOST 处理器 y 和 PCI 设备也可以通过非透明 PCI 桥，直接访问 PCI 总线 x 域的地址空间，并通过 HOST 主桥 x 访问存储器 x。为此非透明 PCI 桥需要对分属不同 PCI 总线域的地址空间进行转换。

目前有许多厂商可以提供非透明 PCI 桥的芯片，在具体实现上各有差异，但是其基本原理类似，下面以 Intel 21555 为例说明非透明 PCI 桥。值得注意的是，在 PCIe 体系结构中，也存在非透明 PCI 桥的概念。

2.5.1 Intel 21555 中的配置寄存器

Intel 21555 非透明 PCI 桥源于 DEC21554^①，并在此基础上做了一些改动。Intel 21555 桥片与其他透明桥在系统中的位置相同。如图 2-16 所示，这个桥片一边与 Primary PCI 总线相连，另一边与 Secondary PCI 总线相连。

在 Intel 21555 桥片中，包含两个 PCI 设备配置空间，分别是 Primary PCI 总线配置空间和 Secondary PCI 总线配置空间，处理器可以使用 Type 00h 配置请求访问这些配置空间。在大多数情况之下，在 Primary PCI 总线上的 HOST 处理器管理 Primary PCI 配置空间；在 Secondary PCI 总线上的 HOST 处理器管理 Secondary PCI 配置空间^②。

在 Intel 21555 桥片中，还有一组私有寄存器 CSR（Control and Status Register），系统软件使用这组寄存器对非透明桥进行管理并获得桥片的一些信息，这组寄存器可以被映射成为 PCI 总线的存储器地址空间或者 I/O 地址空间。

本章仅介绍 Primary PCI 总线这一边的配置寄存器，Secondary PCI 总线的配置寄存器虽然与 Primary PCI 总线的这些寄存器略有不同，但是基本对等，因此本节对此不做介绍。Primary PCI 总线的配置寄存器如表 2-6 所示。

表 2-6 Primary PCI 总线的配置寄存器

地 址 偏 移	寄 存 器	PCI 配置寄存器	复 位 值
0x13~0x10	Primary CSR and Memory 0 BAR	BAR0	0x0000-0000
0x17~0x14	Primary CSR I/O BAR	BAR1	0x0000-0001
0x1B~0x18	Downstream Memory 1 BAR	BAR2	0x0000-0000
0x1F~0x1C	Downstream Memory 2 BAR	BAR3	0x0000-0000
0x23~0x20	Downstream Memory 3 BAR	BAR4	0x0000-0000
0x27~0x24	Downstream Memory 3 Upper 32 Bits	BAR5	0x0000-0000
0x97~0x94	Downstream Memory 0 Translated Base	None	不确定
0x9B~0x98	Downstream I/O or Memory 1 Translated Base	None	不确定
0x9F~0x9C	Downstream Memory 2 Translated Base	None	不确定
0xA3~0xA0	Downstream Memory 3 Translated Base	None	不确定

① DEC21554 是 Digital 公司的产品。

② Intel 21555 非透明桥片两边的 HOST 处理器都可以访问 Primary 和 Secondary 总线的配置寄存器。

从表 2-6 中，我们可以发现 Primary PCI 总线的这些配置寄存器共分为两组，一组寄存器与 PCI 设备的配置寄存器的 BAR0~5 对应，这些寄存器与标准 PCI 配置寄存器 BAR0~5 的功能相同；另一组寄存器是 Translated Base 寄存器，这组寄存器的主要作用是将来自 Primary PCI 总线的数据访问转换到 Secondary PCI 总线。

其中 BAR0~5 寄存器在系统初始化时由 Primary PCI 总线上的 HOST 处理器进行配置，配置过程与 PCI 总线上的普通设备完全相同。只是 Intel 21555 规定，BAR0 只能映射为 32 位存储器空间。

CSR 寄存器可以根据需要映射在 BAR0 空间中，此时 BAR0 空间最小为 4 KB。CSR 寄存器也可以根据需要使用 BAR1 寄存器映射为 I/O 地址空间，同时 BAR1 寄存器还可以映射其他 I/O 空间；BAR2~3 只能映射为 32 位存储器地址空间；而 BAR4~5 用来映射 64 位的存储器地址空间。

对于 Primary PCI 总线，所有 BAR0~5 寄存器映射的地址空间都将占用 Primary PCI 总线域，然而这些地址空间中所对应的数据并不在 Primary PCI 总线域中，而是在 Secondary PCI 总线域中。Translated Base 寄存器实现不同 PCI 总线域地址空间的转换，Intel 21555 将不同 PCI 总线域地址空间的转换过程称为“地址翻译”。

Intel 21555 支持两种地址翻译方法，一种是直接地址翻译，另一种是查表翻译。Primary PCI 总线的 BAR 空间只支持直接地址翻译，而 Secondary PCI 总线的 Memory 2 BAR 空间支持查表翻译，本节仅介绍直接地址翻译方法，对查表翻译有兴趣的读者请阅读 Intel 21555 的数据手册^①。直接地址翻译过程如图 2-17 所示。

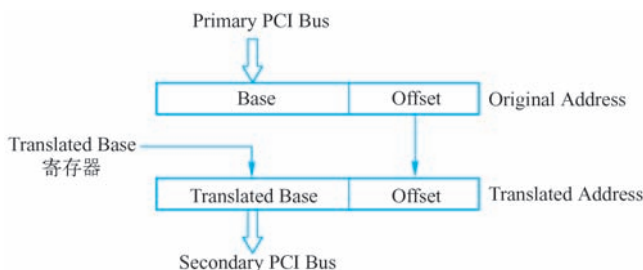


图 2-17 Intel 21555 的直接地址翻译过程

当 Primary PCI 总线对非透明桥 21555 的 BAR0~5 地址空间进行数据请求时，这个数据请求将被转换为对 Secondary PCI 总线的数据请求。Translated Base 寄存器将完成这个地址翻译过程，下节将结合实例说明这个直接地址翻译过程。

2.5.2 通过非透明桥片进行数据传递

下面以图 2-16 中处理器 x 访问处理器 y 存储器地址空间的实例，说明非透明桥 21555 如何将 PCI 总线 x 域与 PCI 总线 y 域联系在一起。

处理器 x 在访问处理器 y 的存储器空间之前，需要做一些必要的准备工作。

① 多数半导体厂商提供两类芯片手册，分别是 Datasheet 和 User manual。其中 Datasheet 偏重硬件电气特性，User Manual 偏重芯片使用原理。

1) 首先确定由哪一个 BAR 寄存器空间映射处理器 y 的存储器地址空间。本节假定使用 BAR2 寄存器映射处理器 y 的存储器地址空间。

2) BAR2 寄存器使用 Downstream Memory 2 Translated Base 寄存器，将来自 Primary PCI 总线的访问转换为对 Secondary PCI 总线地址空间的访问。其中 Downstream Memory 2 Translated Base 寄存器可以由处理器 x 或者处理器 y 根据需要进行设置。

假定处理器 x 和 y 的 HOST 主桥使用“直接相等”策略，建立存储器域与 PCI 总线域间的映射；而处理器 x 使用 BAR2 地址空间访问处理器 y 存储器空间 0x1000-000~0x1FFF-FFFF；处理器 x 的系统软件事先将 BAR2 寄存器设置完毕。处理器 x 访问处理器 y 的这段存储器空间的步骤如下，读者可参考图 2-18 理解以下步骤。

1) 首先处理器 x 访问在处理器 x 域中，且与非透明桥的 BAR2 空间相对应的存储器地址空间。

2) HOST 主桥将进行存储器域到 PCI 总线域的转换，并将这个请求发送到 Primary PCI 总线上。

3) 非透明桥发现这个数据请求发向 BAR2 地址空间，则接收这个数据请求，并在桥片中暂存这个数据请求。

4) 非透明桥根据 Downstream Memory 2 Translated Base 寄存器的内容，按照图 2-17 所示的规则进行地址转换。假设 Downstream Memory 2 Translated Base 寄存器的基地址被预先设置为 0x1000-0000，大小为 256 MB（这个物理地址属于处理器 y 的主存储器地址空间）。

5) 经过非透明桥的转换后，这个数据请求将穿越非透明桥，从 Primary PCI 总线域进入 Secondary PCI 总线域，然后访问处理器 y 的基地址为 0x1000-0000 的存储器区域。

6) 处理器 y 的 HOST 主桥接收这个存储器访问请求，并最终将数据请求发向处理器 y 的存储器中。

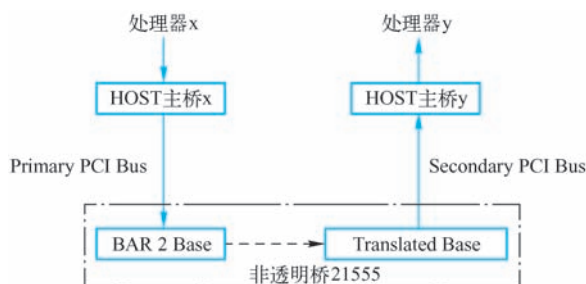


图 2-18 通过非透明桥 21555 进行数据传递

非透明桥 21555 除了可以支持存储器到存储器之间的数据传递，还支持 PCI 总线域到存储器域，以及 PCI 总线域之间的数据传递，此外非透明桥 21555 还可以通过 I²O 和 Doorbell 寄存器进行 Primary PCI 总线与 Secondary PCI 总线之间的中断信号传递。本节对这部分内容不做进一步介绍。

非透明桥有效解决了使用 PCI 总线连接两个处理器存在的问题，因而得到了广泛的应用。在 PCIe 体系结构中，也存在非透明 PCI 桥的概念。如在 PLX 的 Switch 芯片中，各个端口都可以设置为非透明模式。

2.6 小结

本章介绍了在 PCI 总线中使用的桥，包括 HOST 主桥和 PCI 桥，并较详细地介绍了如何使用这些桥访问 PCI 设备的配置空间。

其中 HOST 主桥并不在 PCI 总线规范的约束范围内，不同的处理器可以根据需要设计出不同的 HOST 主桥。本章更加侧重介绍 PowerPC 处理器使用的 HOST 主桥，在该主桥的设计中，提出了许多新的概念，并极大促进了 PCI 总线的发展，在这个桥片中出现的许多新的思想被 PCI V3.0 总线规范采纳。

在 PowerPC 处理器的 HOST 主桥中，明确了存储器域与 PCI 总线域的概念。而区分存储器域与 PCI 总线域也是本章的书写重点，本书将始终强调这两个域的不同。有些处理器系统并没有明确区分这两个域的差别，因此许多读者忽略了 PCI 总线域的存在，并错误地认为 PCI 总线域是存储器域的一部分。

本章还重点介绍了 PCI 桥的实现机制。在许多较简单的处理器系统中，并不包含 PCI 桥，但是读者仍然需要深入理解 PCI 桥这一重要概念。深入理解 PCI 桥的运行机制，是理解 PCI 体系结构的重要基础。